Data Exploration and Preparation — Commentary

Lesson 2 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

Not examinable · reading time about 90 minutes

Contents

Part I — Opening and the dataset **3**
Slide 1 — Lesson 2: Data Exploration and Preparation . . . 3
Slide 2 — Before we start . . . 4
Slide 3 — What today builds on . . . 4
Slide 4 — Today . . . 4
Slide 5 — One dataset, all lesson long . . . 4

Part II — Exploration and missing values **5**
Slide 6 — Look before you touch anything . . . 5
Slide 7 — Outliers squash two of these four . . . 5
Slide 8 — Where the gaps are . . . 6
Slide 9 — Three reasons a value is missing . . . 7
Question — the support-calls story . . . 7
Question — zero calls in the CRM . . . 8
Question — an example of MNAR . . . 8
Slide 10 — age: missing completely at random . . . 9
Slide 11 — What mean imputation actually costs . . . 9
Worked example — four ages out of ten . . . 11
Slide 12 — Support calls: missing at random . . . 11
Question — what links `age_imputed` to `age_missing`? . . . 12
Question — how can the model tell which value was missing? . . . 12
Slide 13 — What to do about a gap . . . 13
Question — why is the indicator for MAR and not MCAR? . . . 13
Slide 14 — Choosing among the four . . . 14

Part III — Outliers **15**
Slide 15 — Extreme does not mean wrong . . . 15
Question — only numeric features? . . . 15
Slide 16 — Rule 1: distance in standard deviations, flag beyond k = 3 . . . 15
Worked example — the ruler made of the outlier . . . 16
Slide 17 — Rule 2: interquartile range (IQR) . . . 16
Worked example — the same eight values . . . 17
Slide 18 — Close fences, very different counts . . . 17

Slide 19 — Quartile rule: 12 false alarms . . . 18
Slide 20 — What neither rule can tell you . . . 18
Slide 21 — Once something is flagged . . . 19
Slide 22 — Notebook 1, live . . . 19
Slide 23 — Break . . . 19

Part IV — Scaling **19**
Slide 24 — Why scale? . . . 19
Slide 25 — Unequal scales: a narrow valley . . . 20
Question — where the paraboloid comes from . . . 21
Question — empirical risk and cost function . . . 21
Slide 26 — A 100:1 variance ratio, and what it costs . . . 22
Worked example — one step size for two directions . . . 23
Slide 27 — StandardScaler vs MinMaxScaler . . . 23
Worked example — five customers, one outlier . . . 24

Part V — Encoding categories **24**
Slide 28 — Every encoding makes a claim about the categories . . . 24
Slide 29 — One customer, three encodings . . . 24
Worked example — ten customers, three encodings . . . 25
Slide 30 — Ordinal: equal steps, unequal churn . . . 25
Worked example — the steps ordinal encoding assumes equal . . . 26
Slide 31 — One-hot: no order assumed, but wide . . . 26
Slide 32 — One one-hot column too many . . . 26
Question — why a column of ones? . . . 26
Slide 33 — 4 columns, but only rank 3 . . . 27
Worked example — four rows, four columns, rank three . . . 28
Slide 34 — The curse of dimensionality, briefly . . . 28
Slide 35 — zip_code: 493 codes, 2,000 customers . . . 29
Question — about 100 codes with 4 customers? . . . 29
Question — “a quarter of the statistic” . . . 29
Question — the notebook’s zip-code summary . . . 30

Part VI — Pipelines and feature engineering **30**
Slide 36 — Preprocessing is part of f . . . 30
Slide 37 — ColumnTransformer + Pipeline . . . 31
Slide 38 — The code . . . 31
Question — mean or median, and why the median here . . . 31
Slide 39 — The model, on churn . . . 32
Worked example — the AUC by counting pairs . . . 33
Slide 40 — Where the errors fall . . . 33
Slide 41 — Feature engineering . . . 34
Slide 42 — Notebook 2, live . . . 34

Part VII — Leakage in preprocessing **35**
Slide 43 — Same rule, broken three ways . . . 35
Slide 44 — Leak 1: impute before splitting . . . 35
Worked example — the two nearest neighbours are both in the test set . . . 36

Slide 45 — The smoking gun . . . 36
Slide 46 — Leakage, counted . . . 36
Slide 47 — Leak 2: encode before splitting . . . 37
Slide 48 — A column with no real signal, encoded three ways . . . 38
Slide 49 — How much of its own label a row sees . . . 38
Worked example — a zip code with no signal, and an AUC of 0.875 . . . 39
Slide 50 — The leak shrinks as 1/n_c . . . 40
Slide 51 — The fix . . . 40
Worked example — the same eight customers, cross-fitted . . . 41
Slide 52 — The rule, restated . . . 41
Slide 53 — Notebook 3, live . . . 41

Part VIII — Closing **42**
Slide 54 — What we did today . . . 42
Slide 55 — Homework: we discuss it on Friday 16 October . . . 42
Slide 56 — Before next week . . . 42

What this document is. A companion to the slides of lesson 2: for every slide, what it means, the steps it leaves implicit, and the questions it tends to raise, answered under the slide they belong to. It is a second explanation of the same material, in a different voice from the handout, which remains the reference for the derivations. It is **not examinable**: nothing here is needed for the exam that is not already in the handout and the notebooks.

Slide numbers and titles are the ones in the lesson's slide deck, in `Slides/`. Every number quoted from the lesson's data comes from a committed notebook output.

Worked examples sit under twelve slides (11, 16, 17, 26, 27, 29, 30, 33, 39, 44, 49, 51): a handful of invented values each, small enough to work through by hand. Their numbers are not the lesson's data; every one is computed by `commentary_examples.py`, beside this file, which also writes them here.

Part I — Opening and the dataset

Slide 1 — Lesson 2: Data Exploration and Preparation

The title slide, but it sets the subject well. The lesson does not yet study a particular model in depth. It studies **what has to happen before a model can be trained correctly**.

The link with lesson 1 is the thread of the whole lesson. Lesson 1's rule was *nothing is learned before the split*. Lesson 2 discovers that the model is not the only thing that learns: an imputer, a scaler and some encoders also compute quantities from the data. Data preparation therefore becomes part of learning, and falls under the same rule.

Slide 2 — Before we start

The slide opens the discussion of Exercise 1, on wine quality. What matters is not the accuracy a student reached but **how they set up the problem**. Wine quality could be treated as:

- **classification**, turning the score into "good" and "not good";
- **regression**, keeping the numerical score.

Both are defensible. What counts is being able to say **why** one was chosen. That matters because at the exam one of the ten exercises is drawn and you talk through your own notebook. The course is not teaching a sequence of scikit-learn calls; it is teaching the reasoning behind them.

Slide 3 — What today builds on

One of the most important slides of the introduction. Lesson 1's rule was: split first, then learn from the data. Today's novelty is how wide the word **learn** turns out to be:

- a `StandardScaler` computes a mean and a standard deviation;
- a `SimpleImputer` may compute a median;
- a `KNNImputer` keeps the training rows to find neighbours in;
- a target encoder computes averages of the target.

All of these become part of the predictive system. That is why `Pipeline` stops being "the convenient way to put the scaler before the model" and becomes **the mechanism that enforces the train/test separation in the code**, instead of relying on the programmer to remember it.

Slide 4 — Today

The map of the lesson: exploratory analysis → missing values → outliers → scaling and encoding → feature engineering and pipelines → leakage.

These are not six independent topics. They all lead to the last one: **many harmless-looking preprocessing steps learn something from the data, and can therefore leak**. The leakage section is not an extra topic added at the end; it is where the whole lesson is going.

Slide 5 — One dataset, all lesson long

2,000 customers of a telecom company. The task is to predict **churn**: whether a customer cancels their contract. The numeric features are tenure, monthly charges, age and support calls; the categorical ones are contract type, region and `zip_code`, which has **493 different values**.

The dataset is synthetic, and that is a deliberate choice rather than a weakness. Because we know the process that generated it, we know which columns really carry information about churn and which do not. In particular, **`zip_code` has no relation to churn by construction**. This becomes decisive later: when target encoding appears to extract a large signal from `zip_code`, we know for certain that the signal is manufactured. The generator confirms it: churn depends on tenure, contract type, monthly charges and support calls, while `zip_code` and region are noise with respect to the target.

The business context: the model decides which customers the retention team should call. A false positive costs a useless phone call; a false negative may cost a customer. This asymmetry returns in lesson 4.

Part II — Exploration and missing values

Slide 6 — Look before you touch anything

The first practical rule: **look at the data before transforming it**, with `.info()`, `.describe()` and the share of each class.

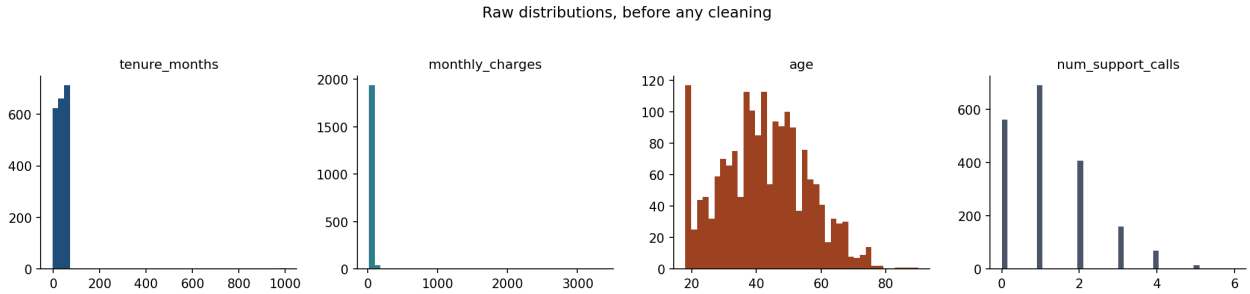
Three numbers matter: 2,000 rows, 8 columns, a churn rate of 19.4%. A classifier that always predicts "no churn" is therefore already right

$$1 - 0.194 = 0.806$$

of the time: **80.6% accuracy** for doing nothing. That is the baseline, and it is why a model at 81% should impress nobody.

The second alarm is the maximum of `tenure_months`: 999 months, about 83 years. It is not a typo on the slide; the data really contains impossible values.

Slide 7 — Outliers squash two of these four



Every numeric column, before anything is done to it. What to look at is what is ***missing****:* ***tenure_months*** *runs to 1000 and* ***monthly_charges*** *to 3500 with no visible bar anywhere out there, and both columns are crushed into a single spike at the left. That empty stretch of axis is the whole finding — a handful of values so extreme they have flattened the shape of everything else. Compare* ***age*** *and* ***num_support_calls****, drawn on their own honest range, which actually show a distribution.*

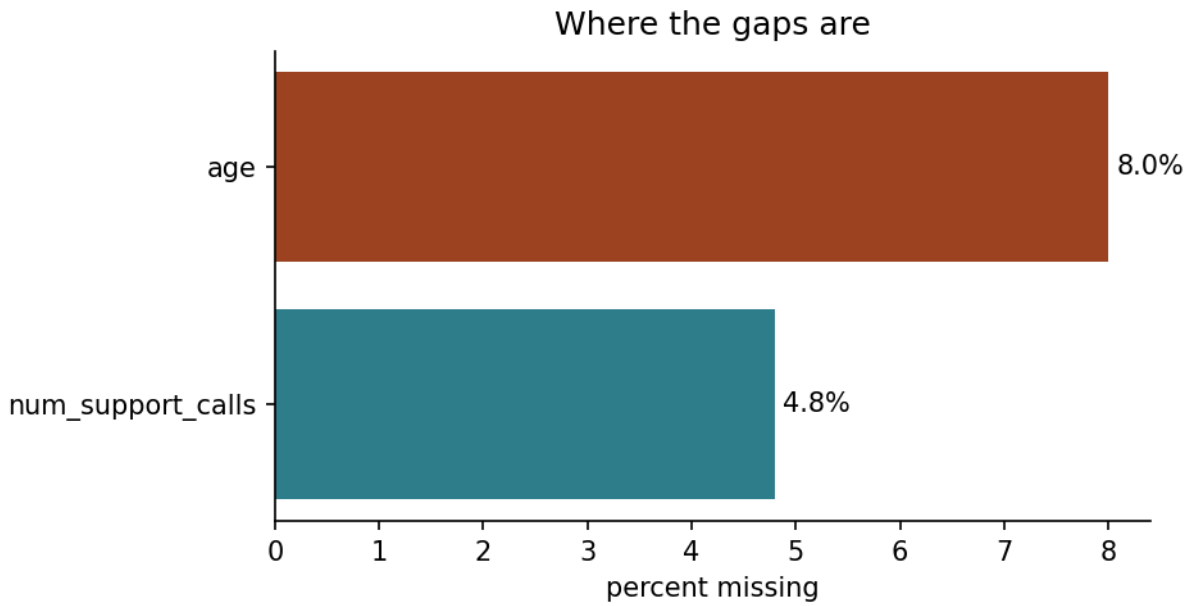
Start from the figure: four histograms, of `tenure_months`, `monthly_charges`, `age` and `num_support_calls`.

For the first two, **do not try to describe a shape**, because there is none to see. The axis of `tenure_months` has to reach about 1,000 although almost every customer sits between 0 and 72; `monthly_charges` contains values above 3,000 while ordinary charges are in the tens. The ordinary

customers are squashed into a single bar at the left. `age` and `num_support_calls`, on realistic ranges, do show their distributions.

The message is not only "outliers can disturb the model". Here they are already disturbing **us**: they stop us reading the dataset at all.

Slide 8 — Where the gaps are

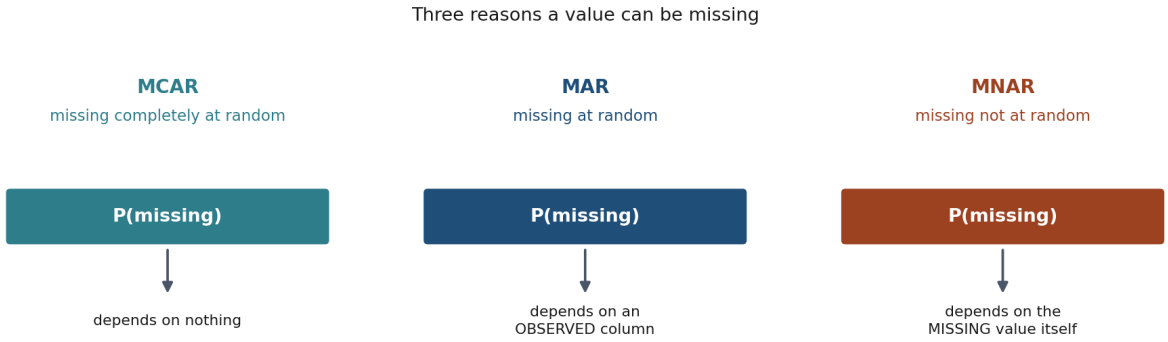


Where the gaps are. What a bar chart of missingness cannot show is whether the gaps are related to each other or to the target — which is exactly what decides the fix.

The chart shows, for each column with gaps, the percentage missing: `age` 8.0%, `num_support_calls` 4.8%. The first thing to learn about missing data is **where it is and how much of it there is**.

The slide also states the chart's limit: it cannot say **why** the values are missing. And the why decides the treatment. Two columns each missing 5% of their values may need completely different handling.

Slide 9 — Three reasons a value is missing



The three mechanisms side by side. The distinction is not academic: it determines which repair is valid and which quietly reweights your sample.

The three mechanisms, each named by what the probability of a value being missing depends on.

Missing completely at random (MCAR). The probability of a gap depends neither on the missing value nor on anything else we hold. The field was left blank for a reason unrelated to the data. In this dataset `age` is built this way.

Missing at random (MAR). The name is misleading: it does **not** mean the gaps are unrelated to everything. It means the probability of a gap can be explained by something we **do observe**. In this dataset `num_support_calls` is missing more often for long-tenure customers, so the gaps depend on `tenure_months`, which is in the table.

Missing not at random (MNAR). The probability of a gap depends on the missing value itself, even after accounting for the other columns. The dataset alone cannot usually reveal this; it takes knowledge of how the data was collected.

The intuition to leave with the class:

- **MCAR**: the gap tells you nothing;
- **MAR**: the gap can be explained by columns you have;
- **MNAR**: the gap depends on something you cannot see.

Question — the support-calls story

> *`num_support_calls` is said to be missing more often for long-tenure customers because part of their history predates the company's current customer system (CRM). But even if part of the history predates the CRM, could there not have been later support calls?*

Yes, and the objection is right: the story is a simplification worth making precise. It depends on what `num_support_calls` means.

If it means **the total number of support calls over the customer's whole relationship with the company**, then for a long-tenure customer part of that relationship predates the current CRM. There may well be calls recorded since the CRM was introduced. For example: a customer for eight years, a CRM introduced three years ago, two calls recorded in those three years, and nothing known about the first five. The total is then not 2; it is **unknown**, even though part of it is recorded, and `NaN` is the honest representation:

calls recorded since the CRM $\neq$ calls over the whole relationship.

That is what makes the mechanism MAR: the probability that the count is incomplete grows with an observed column, tenure.

If instead the column meant "calls in the last twelve months", the story would not work at all: calls before the CRM would be irrelevant and the recent count would be complete. So a more precise version is:

> `num_support_calls` is the total number of support calls over the customer's recorded lifetime. For long-tenure customers part of that lifetime predates the current CRM, so the complete count may be unavailable, and a gap becomes more likely as tenure grows. This does not mean there are no recent records; it means the value the feature is meant to hold cannot be reconstructed.

In the synthetic dataset itself, the probability of a gap is simply made to grow with `tenure_months`; no sequence of calls before and after a CRM is simulated. The CRM story is the intuition for the MAR mechanism, not a model of an information system.

Question — zero calls in the CRM

> *So even if the CRM shows 0 calls, do we not know what happened in the first years?*

Right, provided it is clear that the 0 refers only to the part of the history the current CRM can see: "for a long-tenure customer, even if the current CRM shows no support calls, we cannot conclude they never called; there may have been calls before the CRM existed, so the complete count is unknown." It is also why the value is `NaN` and not `0`:

- `0` means **we know the total number of calls is zero**;
- `NaN` means **we do not know the total number of calls**.

Question — an example of MNAR

> *The handout illustrates MNAR with customers who receive a very high bill and abandon a questionnaire. What does that mean, and is there an example using the lesson's own dataset?*

The handout's example is this: a customer receives an unusually high bill, decides to cancel, and for that reason does not complete a questionnaire, leaving some of its fields empty. The probability that those fields are missing is linked to something unobserved that also caused the cancellation. A more direct version is a questionnaire asking "how much was your last bill?", where customers with very high bills are more likely not to answer: the probability that the answer is missing depends on the very value that is missing. Note that no bill column exists in the lesson's dataset; the example is a story, not a feature.

The lesson's dataset contains **no MNAR column**: `age` is MCAR and `num_support_calls` is MAR. The cleanest example therefore uses a real column with a hypothetical mechanism, `age`:

> In our data `age` is MCAR: some customers left it blank for reasons unrelated to their age. Now imagine instead that older customers are more reluctant to state their age. Then the probability that `age` is missing would depend on the age we cannot see. That would be MNAR.

true age	`age` as recorded
25	25
38	38
51	51
72	NaN
81	NaN

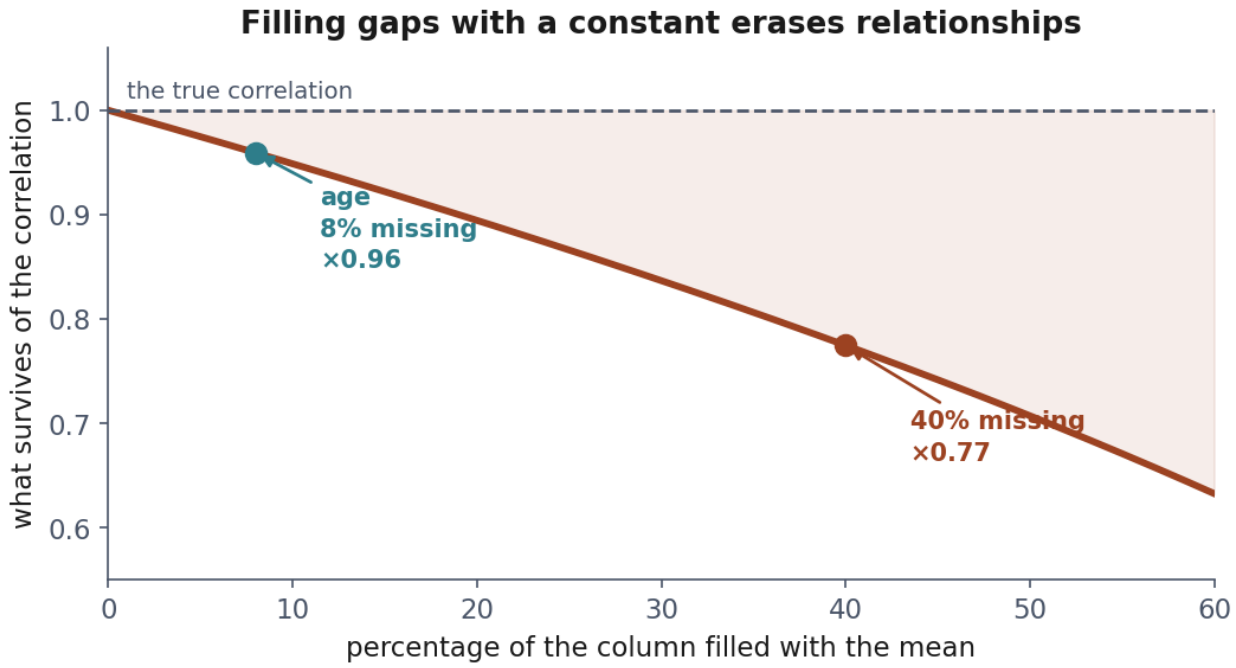
Formally, $P(\text{age missing} \mid \text{age} = 80) > P(\text{age missing} \mid \text{age} = 30)$. The consequence is that the recorded ages look younger than the population really is, and no other column can correct it, because the cause of the gap is the missing value itself. Comparing MCAR and MNAR on the same column makes the difference immediate.

Slide 10 — age: missing completely at random

The slide builds a deliberately simple question: **if `age` is MCAR, is filling its gaps with the mean free?** The intuitive answer is yes, because the mean of the observed values is an unbiased estimate of the true mean, and the mean of the column does stay correct.

But the column's statistical structure does not stay the same. Filling many gaps with the mean piles observations exactly at the centre of the distribution, which shrinks the variance and weakens every relationship `age` has with other columns. The slide is the set-up for the derivation on the next one; the misconception is stated on purpose and corrected there.

Slide 11 — What mean imputation actually costs



What filling with the mean actually costs, plotted as the fraction of a true correlation that survives. What to look at is that the curve starts at 1 and only ever falls: there is no missing fraction at which mean imputation is free. The two markers are the cases worked through below — `age` at 8% missing keeps 96% of its relationships, which is why nobody notices, and a column at 40% keeps 77%, which erases a quarter of a genuine signal without touching the column's own mean.

Read the chart this way. The horizontal axis is the percentage of the column replaced by the mean; the vertical axis is **how much of the original correlation survives**. The line starts at 1 when nothing is filled and falls as $\sqrt{1-p}$: no positive amount of mean imputation leaves the correlation untouched.

Why it happens. Centre X so that its mean is 0. After mean imputation

$$X' = \begin{cases} X & \text{with probability } 1-p \\ 0 & \text{with probability } p. \end{cases}$$

The filled values sit exactly on the mean and contribute nothing to the spread, so

$$\operatorname{Var}(X') = (1-p)\operatorname{Var}(X), \qquad \sigma_{X'} = \sqrt{1-p}\,\sigma_X.$$

The covariance with any Y loses the contribution of the filled rows in the same proportion: $\operatorname{Cov}(X', Y) = (1-p)\operatorname{Cov}(X, Y)$. Putting both into the definition of correlation,

$$\rho_{X'Y} = \frac{(1-p)\operatorname{Cov}(X,Y)}{\sqrt{1-p}\,\sigma_X\sigma_Y} = \sqrt{1-p}\,\rho_{XY}.$$

Mean imputation **weakens the signal X carries about Y**. Filling gaps is not neutral bookkeeping: it makes X less able to move together with anything.

Notebook 1's experiment does not impute the churn data progressively. It builds a controlled pair of variables with correlation 0.700, hides a growing fraction of X at random and fills it with the mean. At $p = 0.40$ the formula predicts $\sqrt{0.6} = 0.775$ of the correlation, so $0.700 \times 0.775 = 0.543$; the notebook measures **0.543**. At $p = 0.08$, the fraction of `age` that is really missing, the factor is 0.959 and the correlation 0.671: small enough to ignore.

Worked example — four ages out of ten

Ten customers, ages 24, 28, 31, 35, 38, 42, 45, 49, 52, 56. Lose four of them — 24, 35, 45, 49, a typical draw of four — and fill each with the mean of the six that remain, **41.2**. That is $p = 0.4$.

	before	after imputation	ratio
variance of age	102.0	62.5	**0.61**
correlation with monthly charges	0.860	0.676	**0.79**

The formula says $1 - p = 0.60$ for the variance and $\sqrt{0.6} = 0.77$ for the correlation. The average age barely shifted (40.0 before, 41.2 after, and on average over many draws it does not shift at all), yet about **21%** of the correlation with charges is gone: four points now sit exactly on the average, where they say nothing about how age and charges move together. On ten values the ratios depend on which four go missing; the formula is their average.

Slide 12 — Support calls: missing at random

`num_support_calls` is MAR: long-tenure customers leave it blank more often.

Two consequences. **Dropping the rows with a gap** would remove mostly long-tenure customers, and the remaining sample would no longer represent them. **Filling the gaps** creates a different problem: afterwards the model cannot tell "this customer really had 1 support call" from "this customer had a gap and I wrote 1".

The **missingness indicator** keeps that fact. For a column X_j with gaps, a new binary column is created:

$$M_j = \begin{cases} 1 & \text{if } X_j \text{ was missing} \\ 0 & \text{otherwise.} \end{cases}$$

Every column with gaps gets its own indicator; there is no single generic "was missing" column. With two columns:

age	support_calls	age_missing	support_calls_missing
42	2	0	0
NaN	3	1	0
51	NaN	0	1
NaN	NaN	1	1

In the last row both values are missing, so both indicators are 1. The indicator is **an ordinary column of the design matrix**, not metadata: a linear model gives it a coefficient, a tree can split on it. `SimpleImputer(add_indicator=True)` creates these columns inside the pipeline.

But on this dataset the indicator adds nothing. What it would tell the model, "this is a long-tenure customer", the model already has in `tenure_months`. The handout measures it: adding the indicator moves the area under the receiver operating characteristic curve (AUC) from **0.751 to 0.741**, no gain, well inside the noise of a different split. (The AUC is the probability that the

model scores a randomly chosen churner above a randomly chosen non-churner: 0.5 is a coin flip, 1 a perfect ranking; slide 39 returns to it.) Under MAR the column that explains the gap is in the table by definition.

Question — what links `age_imputed` to `age_missing`?

> *What links the column `age_imputed` to the column `age_missing`? Is there not a danger that the model relates `age_missing` to a different feature?*

The link is made **when the features are built**, and only then: `age_imputed` is `age` with its gaps filled, and `age_missing` records, row by row, whether `age` was blank before filling.

`age` (original)	`age_imputed`	`age_missing`
35	35	0
NaN	42	1
51	51	0
NaN	42	1

After that, **no association between the two columns has to be declared to the model**, and none is. The model receives a vector of features, $(\text{age_imputed}, \text{age_missing}, \text{tenure}, \text{charges}, \dots)$, and learns for itself whether and how to use each. `age_missing` carries what it needs by itself: a 1 in that column means "in this row, `age` was not available". So the model can tell apart the rows $(42, 0)$, an age really equal to 42, and $(42, 1)$, an unknown age filled with 42. In a linear model,

$$\text{score} = \beta_0 + \beta_1 \,\text{age_imputed} + \beta_2 \,\text{age_missing} + \cdots$$

the term β_2 contributes only in the rows where the age was missing.

The model may well find that `age_missing` correlates with another feature, for example with `tenure_months` if older customers skipped the field more often. That is not a mistake or a mismatch: it is a statistical relationship between two features, and it may be exactly the information about the missingness mechanism the indicator was meant to keep.

Question — how can the model tell which value was missing?

> *I do not understand how the model distinguishes* $(42, 0)$ *from* $(42, 1)$*. If another field in the row is missing, the model could relate it to any feature. And since the model does not read column names, a 1 in `age_missing` does not tell it that the missing value was the age. What happens when two values are missing in the same row?*

The model indeed does not read names; names are for us. What it sees is a fixed **position** in the feature vector. Suppose every row, after preprocessing, has the structure $[\text{age}_{imp}, \text{support}_{imp}, \text{age}_{miss}, \text{support}_{miss}]$. To the model this is $[x_1, x_2, x_3, x_4]$:

`age`	`support_calls`	row after imputation
NaN	3	`[42, 3, 1, 0]`
35	NaN	`[35, 2, 0, 1]`
NaN	NaN	`[42, 2, 1, 1]`

age	support_calls	row after imputation
50	4	`[50, 4, 0, 0]`

What makes it work is that x_3 **is, in every row, built only from whether `age` was missing**, and x_4 only from whether `support_calls` was. The model does not need the word "age"; it needs each position to have the same meaning in every row, exactly as it never knows that 42 in the first position means 42 years. A missing `support_calls` cannot switch on x_3, because x_3 is a different column computed from a different source. The pipeline guarantees that the positions stay the same between `fit` and `predict`.

With two gaps in the same row, both indicators are 1, and in a linear model both contribute: for the third row, score $= \beta_0 + 42\beta_1 + 2\beta_2 + \beta_3 + \beta_4$.

The problem described in the question would be real with a **single generic indicator**, "this row has some gap": then a 1 would not say which value was missing. That is precisely why one indicator per column is built. One refinement: in a plain linear model `age_missing` adds a separate term β_3; it does not make the model treat `age_imputed` differently when the age was filled. That would take an interaction term. The slide's message is simpler: the indicator keeps the information "here `age` was absent", which imputation alone would erase.

Slide 13 — What to do about a gap

The four strategies:

- **Drop rows** only if the gaps are few and the mechanism is MCAR. Otherwise certain kinds of customer are removed selectively and the sample stops representing the population.
- **Drop the column** when it is missing so often that what is left is not worth keeping.
- **Impute**: rebuild the value with a mean, a median, the most frequent category (`SimpleImputer`), the nearest neighbours (`KNNImputer`) or a regression.
- **Add a missingness indicator**, but only if *being missing* tells the model something the other columns cannot: typically when values are missing not at random.

A precision: dropping rows by a fixed rule does not learn a number from the data. But every imputation does: as soon as a mean, a median or a set of neighbours is computed, it must be computed on the training data only.

Question — why is the indicator for MAR and not MCAR?

> *Why is the missingness indicator used for MAR and not for MCAR?*

Under MCAR the fact that a value is missing carries no systematic information: $P(\text{age missing})$ depends on nothing, so the 0/1 of `age_missing` is essentially random and cannot help predict churn. The handout's table shows it: customers with `age` missing churn at 17.5%, those with it present at 19.6%, a difference no larger than chance.

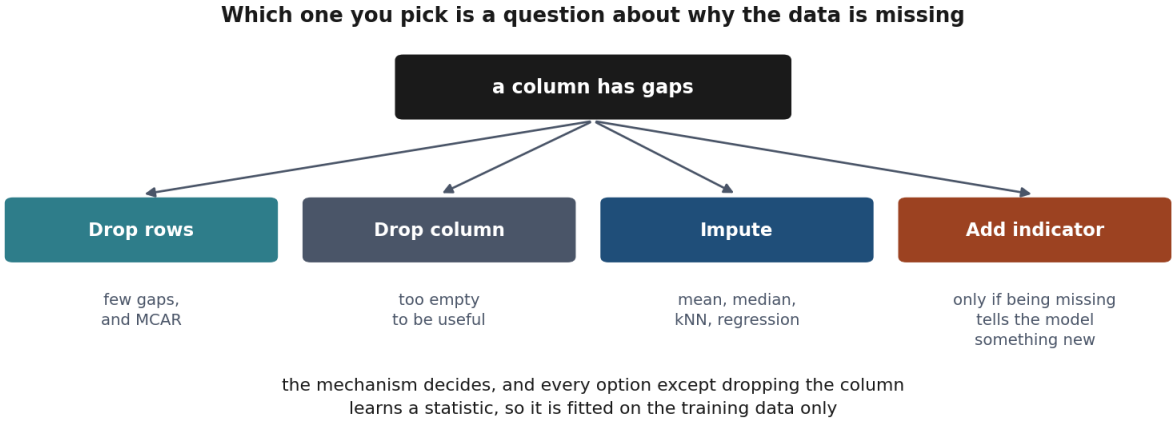
Under MAR the gap is structured. `num_support_calls` is missing for longer-tenure customers: the rows with a gap have a median tenure of 52 months against 35, and churn at 12.5% against 19.7%. So the indicator does say something about the customer.

But that does not make it useful. The structure is explained by `tenure_months`, which is already in the table, so the model can learn "high tenure, more gaps, less churn" without the indicator; adding it moves the area under the curve from 0.751 to 0.741. The rule that survives:

- **MCAR**: the indicator carries nothing, because the gaps are random;
- **MAR**: it carries something, but usually something the explaining column already says;
- **MNAR**: it can carry information no other column has, because the cause of the gap is the missing value itself. It does not solve the MNAR problem, since the value stays unknown; it only keeps the fact that it was missing.

So the slide's rule is not "indicators are for MAR" but "**add one only if being missing tells the model something the other columns cannot**", which is how slides 12 to 14 put it.

Slide 14 — Choosing among the four



The decision, as a chart. Note that every branch depends on the mechanism, which is why Section 3.1 comes first.

The diagram is a decision process, not an algorithm of the kind "below 5%, always do this": how much is missing, why it is missing, how important the column is, and whether being missing tells the model anything new.

The most dangerous branch is often **drop rows**, precisely because it is easy and looks harmless. If the probability of a gap depends on a subgroup, dropping those rows under-represents exactly that subgroup.

The bottom line of the figure is the thread of the lesson: every option except dropping the column learns a statistic from the data, so it is fitted on the training data only.

Part III — Outliers

Slide 15 — Extreme does not mean wrong

The distinction the whole section rests on: **an outlier is not an error**. An extreme value can be a recording error, a rare but genuine value, or a member of a different population. A monthly charge of 3,344.7 may be a typo, or a corporate account that really pays that much; the number alone cannot tell.

An outlier detector answers the question "is this value unusual compared with the others?", never "is this value wrong?". That distinction holds until slide 21.

Question — only numeric features?

Does the concept of outliers apply only to numeric features?

In the sense of slides 15 to 21, yes: the z-score and the interquartile-range rule need a numerical scale on which to measure how far a value is from the rest. A categorical column has no natural distance: nobody can say that `two-year` is farther from `month-to-month` than `one-year` is without imposing a structure.

Categorical columns can still hold anomalous values, but they are better called **rare**, **unexpected** or **invalid** categories: a region spelled `Nroth`, or a category that appears once in 2,000 rows. They are found with frequency counts and domain rules, not with z-scores.

There is also a third case: values that are each normal but unusual together, such as an 18-year-old customer with 70 months of tenure. That is a **multivariate outlier**, and neither rule of this lesson, which look at one column at a time, can see it. In one sentence: the z-score and Tukey's fences are univariate detectors for numerical columns; categorical columns can also contain unusual or invalid values, but they need other checks, such as category frequencies or domain rules.

Slide 16 — Rule 1: distance in standard deviations, flag beyond k = 3

The first rule uses the **z-score**,

$$z_i = \frac{x_i - \bar{x}}{s},$$

and flags a value when $|z_i| > k$, here $k = 3$. The question it asks is: how many standard deviations separate this value from the mean? On a normal distribution $|z| > 3$ is very rare, about 0.27% of values in total.

The weakness is that the ruler, the mean and the standard deviation, is built with the outliers included. A very large value inflates s, and a larger s shrinks its own z-score. This is **masking**: with the charges 40, 45, 50, 55, 60, 65, 70 and a data-entry error of 500, the error inflates the standard deviation so much that its own z-score is only about 2.47, and the 3σ rule does not flag it.

Worked example — the ruler made of the outlier

Eight monthly charges: 40, 45, 50, 55, 60, 65, 70 and a data-entry error, **500**.

- mean $\bar{x} = 110.6$, standard deviation $s = 157.6$ — the seven ordinary values alone have $s = 10.8$;
- $z_{500} = (500 - 110.6)/157.6 = \mathbf{2.47}$ — below 3, so the rule flags nothing.

This is masking in its purest form, and it is not bad luck: with n values no z-score can exceed $(n-1)/\sqrt{n}$, which for $n = 8$ is **2.47**. On a sample this small the 3σ rule cannot fire whatever the data. The error inflated the very ruler it was measured with.

Slide 17 — Rule 2: interquartile range (IQR)

Three terms first. Q_1, the **first quartile**, is the value below which about 25% of the data lies; Q_3, the **third quartile**, the value below which about 75% lies. The interquartile range

$$\mathrm{IQR} = Q_3 - Q_1$$

is the width of the **middle half of the data**. Tukey's fences are

$$[\, Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}\,],$$

and every value outside them is flagged as a candidate.

Why the IQR resists outliers. Moving one or two extreme points much farther out barely moves the 25th and 75th percentiles, whereas the standard deviation uses the distance of every point from the mean, and one huge value weighs a great deal.

Where 1.5 comes from. On a normal distribution $Q_1 = \mu - 0.6745\sigma$ and $Q_3 = \mu + 0.6745\sigma$, so $\mathrm{IQR} = 1.349\sigma$ and the upper fence is

$$Q_3 + 1.5\,\mathrm{IQR} = \mu + 0.6745\sigma + 1.5 \times 1.349\sigma \simeq \mu + 2.698\sigma.$$

So Tukey's rule and the 3σ rule are not unrelated: on a normal column they put their thresholds in a similar place. Similar, but not the same, which is the next slide.

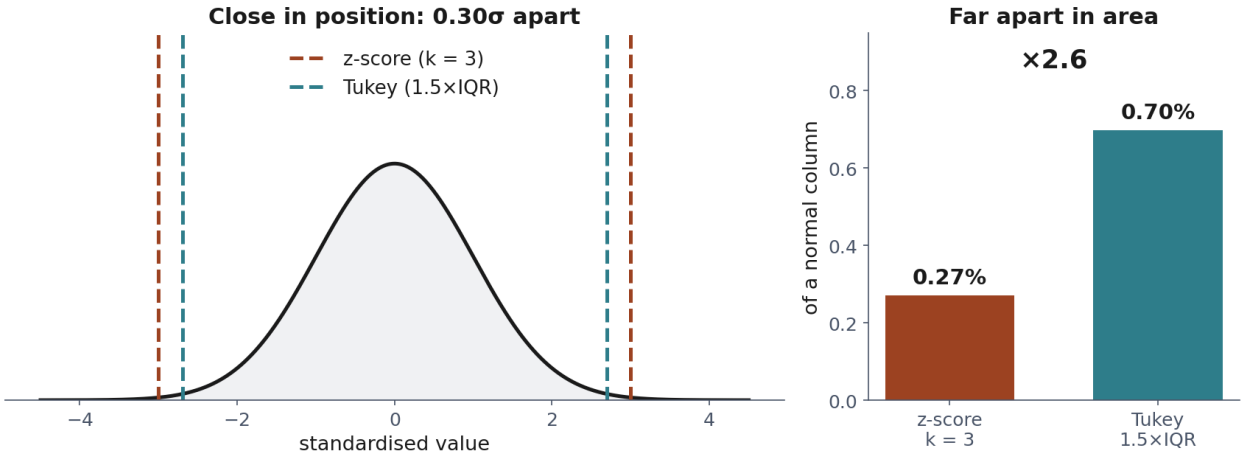
Worked example — the same eight values

Quartiles (NumPy's default, linear interpolation): $Q_1 = 48.75$, $Q_3 = 66.25$, so $IQR = 17.50$ and the fences are

$$Q_1 - 1.5\,IQR = 22.50, \qquad Q_3 + 1.5\,IQR = 92.50$$

500 is far outside: **flagged**, where the z-score saw nothing. The reason is the robustness: drop the 500 and the quartiles of the other seven are 47.50 and 62.50 — the error moved them by a few units, while it multiplied the standard deviation by **14.6**.

Slide 18 — Close fences, very different counts



The two fences, and why "calibrated" is not "agreed". Left: they sit 0.30 standard deviations apart, which is what makes them look interchangeable. Right: the same two rules by how much of a normal column they actually flag — 0.27% against 0.70%, a factor of 2.6. The normal tail falls away so steeply that moving a fence in by a third of a standard deviation nearly triples the area beyond it.

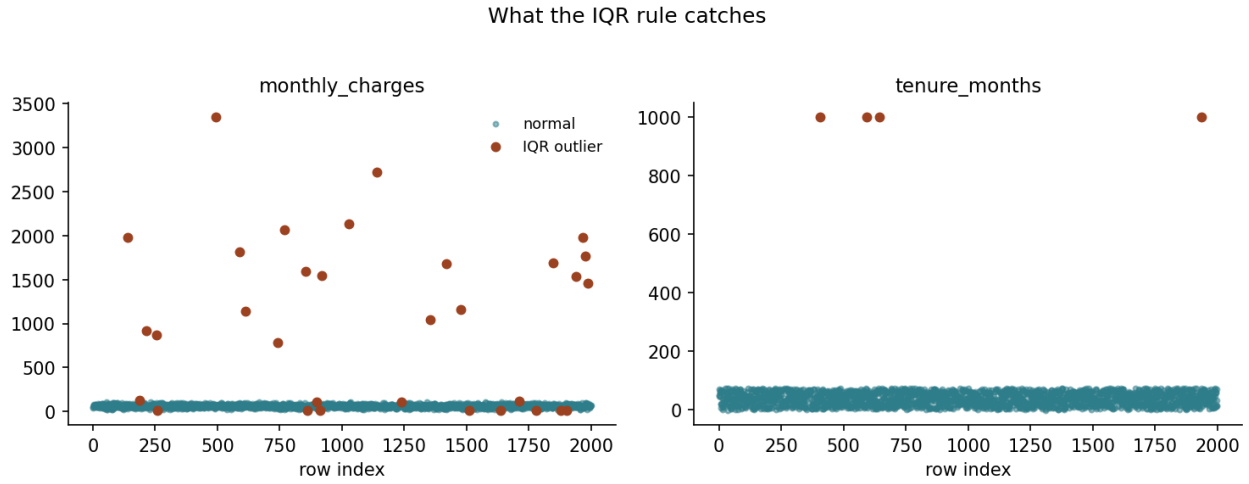
Left: a standard normal distribution, with the fences at $\pm 3\sigma$ and $\pm 2.698\sigma$. They look almost identical, 0.30σ apart.

Right: the share of values beyond each fence,

$$P(|Z| > 3) \simeq 0.27\% \qquad \text{against} \qquad P(|Z| > 2.698) \simeq 0.70\%,$$

a ratio of about 2.6. A threshold that looks nearly the same flags more than two and a half times as many candidates, because in the tails of a normal distribution the density falls very quickly. **Close thresholds do not mean close decisions.**

Slide 19 — Quartile rule: 12 false alarms



Every point Tukey's rule flags in the two contaminated columns. What to look at is the bottom of the left panel: alongside the billing errors up at 800 and beyond, the rule has also flagged a row of perfectly ordinary customers sitting in the main band.

One of the most interesting slides, because it overturns the intuitive conclusion. The data contains 20 genuine billing errors in `monthly_charges`. The z-score rule flags exactly 20; the quartile rule flags 32, the same 20 plus 12 ordinary customers. The z-score seems to have won.

The diagnosis says otherwise. The billing errors inflate the standard deviation from **17.23** (on the 1,980 ordinary customers alone) to **171.25** (on the column as it arrives), dragging the z-score fence from **115.0** out to **593.0**. The detector was badly damaged. It still found every error only because the smallest of them is **780.1**, extreme enough to pass even the ruined fence: **the z-score gave the right answer for the wrong reason.**

The quartile rule kept a robust threshold, with fences at 17.3 and 110.0, but flagged 12 real customers in the tails of the distribution: eight paying 15.0 to 16.7 and four paying 111.6 to 128.4. The conclusion is not "the quartile rule is worse". It is that **a detector's output does not report the detector's health**: the number of points flagged says nothing about whether the rule still works.

Slide 20 — What neither rule can tell you

A `tenure_months` of -3 is impossible, yet it is not statistically extreme for a column spanning 0 to 72. This is the difference between **unusual**, far from the distribution, and **invalid**, impossible in the domain. A statistical rule can find the first; the second needs a **domain rule**, for example $0 \leq$ tenure_months ≤ 120.

Data validation and outlier detection are not the same thing. In notebook 1 the data holds six negative tenures (two at -3 and four at -1), which neither statistical rule flags, while the domain rule flags ten rows: those six plus the four at 999.

Slide 21 — Once something is flagged

Once a candidate is found, the detector's job is over; what happens next is decided by someone who knows the domain. The options: **cap** the value at the rule's limit (*winsorising*: 3,344.7 becomes 110.0); **remove** the row if it belongs to a population we do not want to model; **correct** it if the true value is known; or **leave** it, if it is rare but real.

The distinction to keep is **detection is not decision**. Removing every outlier automatically is dangerous, because it silently changes the population the model is trained on.

Slide 22 — Notebook 1, live

The map of the first notebook, which builds no model yet: pandas and NumPy, used to look at the data and to implement the section's ideas from scratch. It confirms three of them on data:

- the attenuation from mean imputation: a correlation of 0.700 becomes 0.671 at 8% filled and 0.543 at 40%, as $\sqrt{1-p}$ predicts;
- the outlier comparison: z-score 20, quartile rule 32, with the paradox of slide 19;
- the domain check: negative tenures that no statistical rule flags.

The cells headed "Your prediction, before you run the next cell" ask the student to commit to an answer before running the code, which turns the notebook into an experiment rather than a sequence of outputs to read.

Slide 23 — Break

Ten minutes. It also marks the turn of the lesson: until now the data has been **understood and cleaned**; from here it is **turned into a representation a model can use**.

Part IV — Scaling

Slide 24 — Why scale?

The slide introduces gradient descent without yet developing its mathematics. A model holds some numbers to be chosen, its coefficients $w_1, w_2, \ldots$. Every choice of them has an average loss over the training rows, which is lesson 1's empirical risk $\hat{R}_S$. Training means finding the choice that makes it small.

Seen as a function of w_1 and w_2, that average loss is a surface. Lesson 1's slide 20 drew the same idea for a model with a single number, the threshold t: a curve, and learning picked its lowest point. With two coefficients the curve becomes a surface, here a paraboloid.

Gradient descent walks downhill on it. One step is

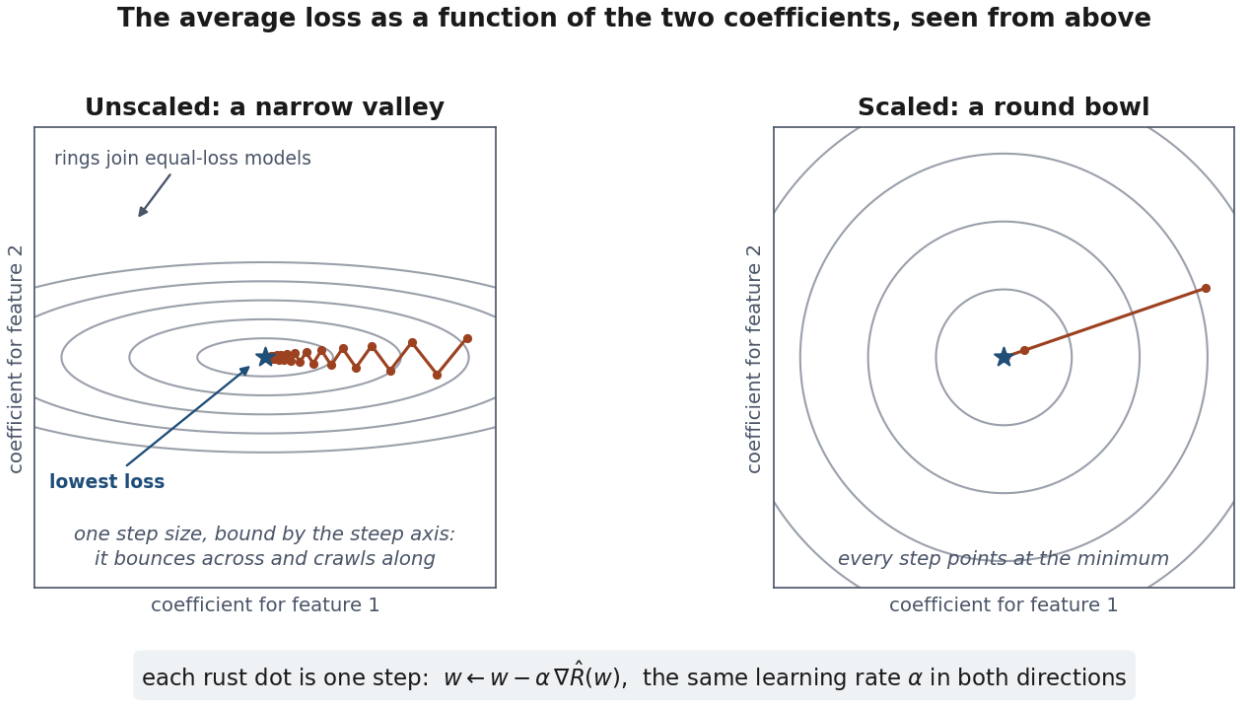
$$w \leftarrow w - \alpha \nabla \hat{R}_S(w),$$

where $\nabla \hat{R}_S(w)$ is the gradient, the direction in which the average loss rises fastest; the minus sign turns it downhill; and α, the **learning rate**, sets how far each step goes. Lesson 3 derives this rule and writes it $\theta \leftarrow \theta - \alpha \nabla J(\theta)$, with θ collecting every parameter, intercept included, and J the cost.

The sentence the next slides illustrate: **one step size has to work in every direction at once.** If one feature lives on a much larger numerical scale than another, the loss reacts very differently to their coefficients, and a single α cannot suit both.

An honest note from the slide's notes: the cleaned churn dataset is **not** a dramatic case; once the outliers are removed its two columns have comparable spreads. That is why notebook 2 builds a toy problem with a variance ratio of about 100:1 to show the effect clearly.

Slide 25 — Unequal scales: a narrow valley



Two features with different variances, before and after scaling. Neither axis is a column of data: each is the coefficient the model gives one feature, so every point in the square is a candidate model, the grey rings join the models that fit equally badly — contour lines of the loss, like altitude on a map — and the star is the model with the lowest loss. The rust dots are successive steps. Left: not a bowl but a ravine — steep across, almost flat along — and one stride has to serve both directions, so the path bounces off the walls while creeping towards the centre; twenty-six steps in, it has still not arrived. Right: scaled, the same problem is round, and every step points at the minimum.

Here it is easy to confuse **features** with **coefficients**, so start from the axes. **The axes are not features.** The horizontal axis is the coefficient w_1, the vertical one the coefficient w_2, so every point (w_1, w_2) is **a different model**. The rings are contour lines of the average loss: each joins models that fit equally badly. The star is the minimum; the rust dots are successive steps of gradient descent, each one $w \leftarrow w - \alpha \nabla \hat{R}(w)$.

Where the features' scales come in. The scales belong to the features, in the data, not to the coefficients. Suppose x_1 takes much larger values than x_2. A small change in w_1 changes the predictions a lot, because $w_1 x_1$ changes a lot; a comparable change through x_2 needs a much larger change in w_2. Seen in the space of coefficients, the loss therefore changes very fast along one direction and very slowly along the other, and the contours become long, thin ellipses. The chain is: 1. different feature scales, so 2. a different sensitivity of the loss to each coefficient, so 3. a different curvature of the loss in each direction, so 4. a narrow valley in the plane of (w_1, w_2).

Without scaling the path bounces between the walls of the valley while creeping along its floor. After scaling the surface is close to a round bowl, and a single α works in both directions.

Question — where the paraboloid comes from

> *Where does the paraboloid come from? Lesson 1 introduced a loss L on each example and its average over the dataset; how does that become a bowl-shaped surface?*

There is a gap, and it closes with a single change of viewpoint that had been left implicit. Lesson 1 wrote the empirical risk as a function of the model f:

$$\hat{R}_S(f) = \frac{1}{m} \sum_{i=1}^{m} L(f(x^{(i)}), y^{(i)}) :$$

a loss for every example, then their average. Slide 25 introduces no new quantity. It draws that same average **as a function of the model's parameters**. If the model has two coefficients, every pair (w_1, w_2) gives a different model, different predictions, different losses and a different average:

$$\hat{R}_S(w_1, w_2) = \frac{1}{m} \sum_{i=1}^{m} L(f_{w_1, w_2}(x^{(i)}), y^{(i)}).$$

This is the formula missing between lesson 1 and slide 25. Lesson 1 asked "how large is the average loss of this model?"; lesson 2 asks "how does that same average loss change as I change the model's parameters?"

Why a bowl. Take a linear model $\hat{y}^{(i)} = w_1 x_1^{(i)} + w_2 x_2^{(i)}$ and the squared loss. One example's loss is $(w_1 x_1^{(i)} + w_2 x_2^{(i)} - y^{(i)})^2$; expanding the square produces terms in w_1^2, w_2^2, $w_1 w_2$, w_1 and w_2, so it is a **quadratic function of the coefficients**. Lesson 1 then averages over the examples, and an average of quadratics is still a quadratic: in two parameters, a paraboloid. Each example contributes an error surface; their average is the total surface.

It also helps to start from one coefficient. With a single w the average loss is a U-shaped curve, and its bottom is the best w; that is lesson 1's search over the threshold. Add a second coefficient and the U becomes a bowl.

Caveat. The paraboloid is exact for a linear model with squared error, which is what the figure draws. The logistic loss of the churn model has that shape only approximately, near its minimum.

Question — empirical risk and cost function

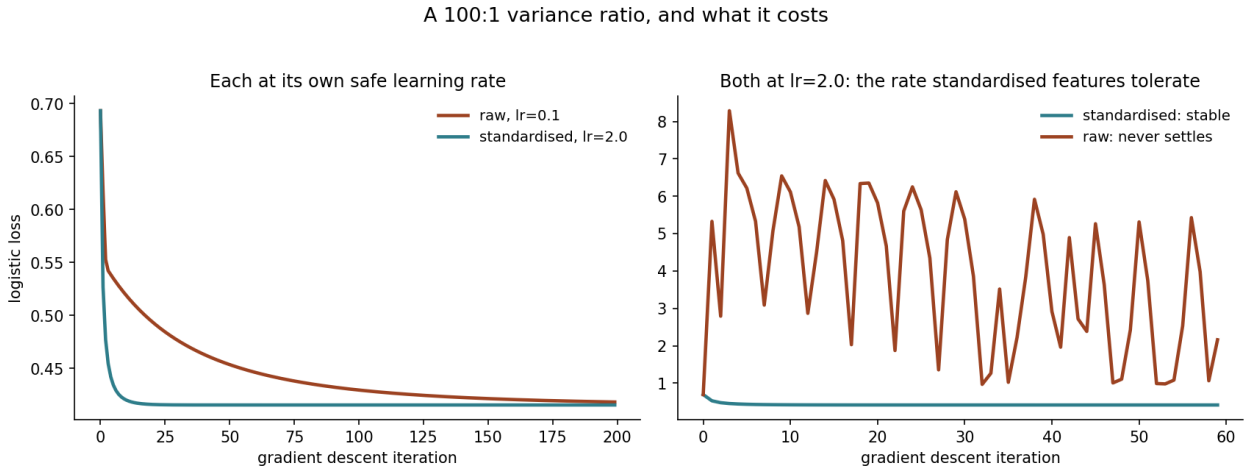
> *Can we say the empirical risk is the cost function we will minimise with gradient descent?*

Yes, in this context. Once the model's parameters are made explicit, lesson 1's $\hat{R}_S$ becomes a function of them, and that function is exactly what gradient descent minimises. The vocabulary

differs by field: statistics calls it the *empirical risk*, optimisation the *cost* or *objective function*. The course writes the cost J from lesson 3 on, and here $J(\theta) = \hat{R}_S(\theta)$. In one sentence: in lesson 1 this quantity was called the empirical risk, the average loss over the training examples; from the point of view of optimisation it is the cost function minimised with respect to the model's parameters.

Two precisions. **Loss and cost are not the same word**: L is the loss on a single example, the cost is the average over the training set. And once **regularisation** arrives in lesson 3, the cost becomes $J(\theta) = \hat{R}_S(\theta) + \lambda\,\Omega(\theta)$, and no longer coincides with the empirical risk alone. For slide 25 the identification is exact.

Slide 26 — A 100:1 variance ratio, and what it costs



The same problem twice, and note the two panels are not the same experiment. Left, each version at the largest rate it tolerates — raw at 0.1, standardised at 2.0 — and after 200 steps the raw run has still not caught up. Right, both forced to the standardised version's rate of 2.0: within 60 steps the raw run is not slow, it is oscillating upwards. Scaling did not just save iterations here; it decided whether the fit finished at all.

The previous slide, measured. The toy problem's two features are built with standard deviations 1 and 10, so the ratio of their variances is $10^2/1^2 = 100$ by construction; on the 1,000 points drawn, notebook 2 measures **110:1**. That is why the title says 100:1 and the notebook's table says 110:1.

With standardised features a learning rate of **2.0** works; the raw features need **0.1**. Give the raw features the rate of 2.0 and the loss does not merely go down more slowly: **it never settles**, swinging between about **0.69 and 8.29** for ever. Note what does not happen: there is no exception and no `NaN`. The program keeps running and looks fine while the optimisation goes nowhere.

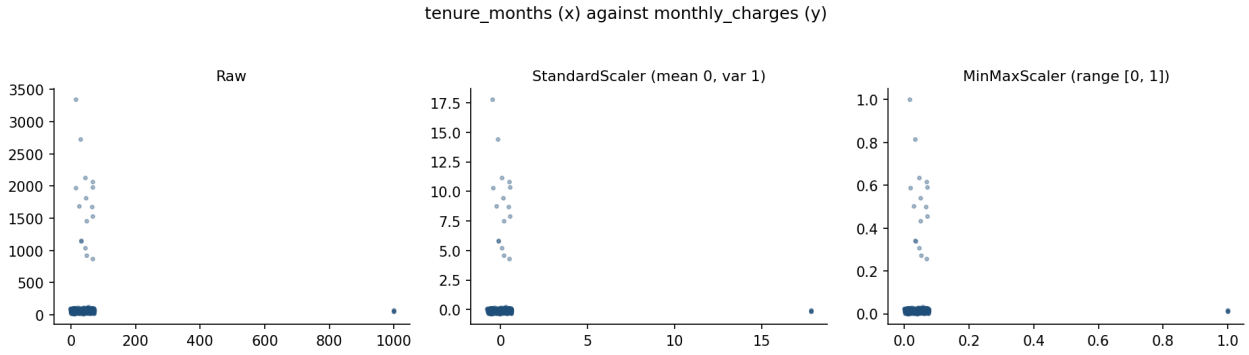
Worked example — one step size for two directions

Take the simplest ravine, $J(w) = \frac{1}{2}(100\,w_1^2 + 1\,w_2^2)$: one direction 100 times steeper than the other, the ratio on the slide. A gradient step multiplies w_1 by $(1 - 100\eta)$ and w_2 by $(1 - \eta)$.

- Stability along the steep axis needs $|1 - 100\eta| < 1$, so $\eta < 2/100 = \mathbf{0.02}$. Above that, w_1 overshoots further every step.
- At $\eta = 0.019$, safely inside, the shallow axis shrinks by only 0.981 per step: **241 steps** to fall to 1% of where it started.

Standardise, so both curvatures are 1, and $\eta = 1$ lands on the minimum in a **single step**. Nothing about the model changed — only the units of the inputs.

Slide 27 — StandardScaler vs MinMaxScaler



`tenure_months` (x) against `monthly_charges` (y), raw and under each scaler. What to look at is the axis numbers, not the cloud — the three panels are the same picture, because scaling moves the ruler and not the data. On the middle panel the largest billing error sits at y 17.8 standard deviations; on the right, min-max is obliged to hand that one error the value 1.0, which leaves every ordinary customer squeezed into the bottom 3% of the axis.

The two transformations:

$$z = \frac{x - \mu}{\sigma} \qquad \text{and} \qquad x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

`StandardScaler` centres the column on 0 with variance 1; `MinMaxScaler` maps the observed values into $[0, 1]$.

The slide's point is their different sensitivity to outliers. If ordinary customers pay 20 to 120 and one value is 3,300, min-max maps 20 to 0 and 120 to about $100/3{,}280 \simeq 0.03$: every ordinary customer ends up in the first 3% of the range. `StandardScaler` is distorted too, since a mean and a standard deviation are not robust, but less directly, because it does not depend on the extreme values alone. Both scalers are fitted on the training data only.

Worked example — five customers, one outlier

Monthly charges 20, 50, 80, 120 and 3300 (mean 714, standard deviation 1293):

charge	MinMax	Standard
20	0.000	-0.54
50	0.009	-0.51
80	0.018	-0.49
120	0.030	-0.46
3300	1.000	2.00

MinMax packs the four ordinary customers into the bottom **3.0%** of $[0, 1]$: 20 and 120 end up 0.030 apart. Standard scaling squeezes them too — all four sit between -0.54 and -0.46 — but they stay distinct on a scale where the outlier is "two units away", not "the whole range".

Part V — Encoding categories

Slide 28 — Every encoding makes a claim about the categories

The starting problem is elementary: a model needs numbers, and `"month-to-month"` is text, so an encoding has to turn categories into numbers. But no encoding is neutral: **each one imposes some structure**. The slide also gives the churn rates the next slides use: month-to-month 0.266, one-year 0.137, two-year 0.071.

Slide 29 — One customer, three encodings

Take a customer on a one-year contract.

- **One-hot**: `[0, 1, 0]`. The only claim is that the three categories are different, none closer to another.
- **Ordinal**: month-to-month 0, one-year 1, two-year 2. This claims an order **and** equal spacing between neighbours.
- **Target**: 0.137, the churn rate of the customer's own category. Compact and apparently informative, but the number was computed from y, and that is where the danger of leakage begins.

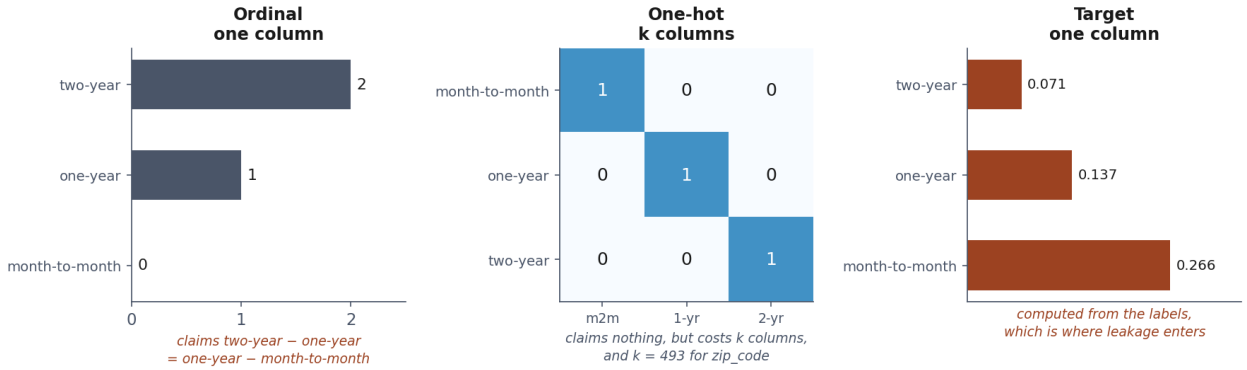
Worked example — ten customers, three encodings

Churn (1) or stay (0), by contract:

contract	labels	one-hot	ordinal	target (churn rate)
month-to-month	1, 1, 1, 0	[1, 0, 0]	0	$3/4 = 0.750$
one-year	1, 0, 0, 0	[0, 1, 0]	1	$1/4 = 0.250$
two-year	0, 0	[0, 0, 1]	2	$0 = 0.000$

One-hot and ordinal are computed from the category alone. The target column is computed **from the labels** — every number in it is an average of y. That is what makes it powerful, and it is the column the second half of the lesson breaks.

Slide 30 — Ordinal: equal steps, unequal churn



`contract_type` under all three encodings. What to look at is the left panel against the right: ordinal spaces the levels 0, 1, 2 — equal steps — while the churn rates it stands in for fall 0.266, 0.137, 0.071, a first step nearly twice the second. The encoding asserts a regularity the column does not have, and a linear model has one coefficient with which to believe it.

Ordinal encoding puts the categories at 0, 1, 2: two steps of exactly 1. The churn rates they stand for fall by $0.266 - 0.137 = 0.129$ and then by $0.137 - 0.071 = 0.066$: the first step is nearly twice the second. The geometry 0, 1, 2 asserts a regularity the data does not have. For a linear model this is binding: a step of $+1$ in the feature always produces the same change, its coefficient. The intuition, without any regression theory: **encode two steps as equal and you ask the model to treat them as equal.** One-hot makes no such claim.

Worked example — the steps ordinal encoding assumes equal

On the ten customers of the previous example the churn rates are 3/4, 1/4, 0. The two steps:

- month-to-month → one-year: $3/4 - 1/4 = 1/2$ (0.500)
- one-year → two-year: $1/4 - 0 = 1/4$ (0.250)

Ordinal encoding gives both steps the same length, 1, so a linear model must predict the same change for both. The data asks for one step twice the other — the same shape as the real rates on the slide, 0.129 and 0.066. One-hot gives each contract its own coefficient and asks nothing.

Slide 31 — One-hot: no order assumed, but wide

A column with k categories becomes k columns of 0s and 1s, with a single 1 per row. It puts the categories in no order and all equally far apart, which is why it is the default. The price is width: for `zip_code`, $k = 493$, so one column becomes 493, almost all zeros. That is what makes target encoding tempting: it turns 493 categories into **one** column. But it uses the target, so it needs great care.

Slide 32 — One one-hot column too many

Take three categories, A, B and C, encoded as d_A, d_B, d_C. Every row belongs to exactly one category, so in every row $d_A + d_B + d_C = 1$. A model with an **intercept** also has, in effect, a column that is always 1, so

$$d_A + d_B + d_C - \mathbf{1} = 0 :$$

an exact **linear dependence** between the columns.

Other numeric features do not remove it. With the columns $[\mathbf{1}, d_A, d_B, d_C, \text{age}, \text{charges}]$ the same combination still gives zero, $-\mathbf{1} + d_A + d_B + d_C + 0 \cdot \text{age} + 0 \cdot \text{charges} = 0$. A set of columns is linearly dependent as soon as **one** non-trivial combination gives zero; the other columns can take part with coefficient 0. Adding features never cures the dummy variable trap.

Question — why a column of ones?

> *Why do "most models include a constant column of ones, the intercept"?*

Two questions in one: what the intercept is, and why it can be written as a column of ones.

What it is. A model that weights two features, score $= w_1 x_1 + w_2 x_2$, is forced to output 0 when both features are 0. Usually we want a starting level independent of the features, so we add a number b: score $= b + w_1 x_1 + w_2 x_2$. That b is the **intercept**: the value the model starts from before adding the contribution of the features. With $b = 5$, a row with all features at zero still scores 5.

Why a column of ones. It is a notational device that lets b be treated like every other coefficient: write b as $b \cdot 1$, put a column that is always 1 in front of the data, and collect b with the other

coefficients:

$$X = \begin{bmatrix} 1 & 10 & 20 \\ 1 & 15 & 30 \\ 1 & 8 & 12 \end{bmatrix}, \qquad w = \begin{bmatrix} b \\ w_1 \\ w_2 \end{bmatrix}, \qquad Xw = \begin{bmatrix} b + 10w_1 + 20w_2 \\ b + 15w_1 + 30w_2 \\ b + 8w_1 + 12w_2 \end{bmatrix}.$$

The column of ones simply makes the same b appear in every row.

Why it matters on this slide. One-hot encode `contract_type`: in every row exactly one of d_M, d_1, d_2 is 1, so their sum is already a column of ones. Add the intercept and you have built the same column twice:

customer	d_M	d_1	d_2	intercept
A	1	0	0	1
B	0	1	0	1
C	0	0	1	1
D	0	1	0	1

$d_M + d_1 + d_2 =$ intercept in every row. The problem is not the intercept itself; it is that the intercept and all k dummies together contain an exact redundancy.

One precision: "most models carry a column of ones" is a teaching shortcut, true of the linear and logistic models the slide is preparing for, not of every algorithm. A decision tree, for instance, has no intercept.

Slide 33 — 4 columns, but only rank 3

Why all k dummies plus an intercept cannot work

contract_type	m2m	1y	2y	sum	intercept
month-to-month	**1**	0	0	**= 1**	**1**
one-year	0	**1**	0	**= 1**	**1**
two-year	0	0	**1**	**= 1**	**1**

the k dummy columns always sum to the intercept column: rank deficient by exactly one

The redundancy made visible: the dummy columns sum to the intercept column, so the design matrix loses rank by exactly one and the solution stops being unique.

The previous slide said there is a redundancy; this one measures it. Three dummies plus the intercept are four columns, but $\text{rank}(X) = 3$: one dimension short, because exactly one relation holds, $d_1 + d_2 + d_3 - \mathbf{1} = 0$.

On the coefficients it looks like this: add a constant c to the intercept, subtract c from **every** dummy's coefficient, and every prediction is unchanged. Infinitely many sets of coefficients are equally good, and none is "the" answer.

The standard fix is to drop one dummy, `OneHotEncoder(drop="first")`. The dropped category becomes the **reference category**, and no information is lost: it is the case in which all the remaining dummies are 0. Two caveats: trees do not suffer from this in the same way, and regularisation can make the solution unique even with redundant columns (lesson 3). For this lesson the point is to see the dependence and the role of the reference category.

Worked example — four rows, four columns, rank three

Four customers, an intercept and all three dummies:

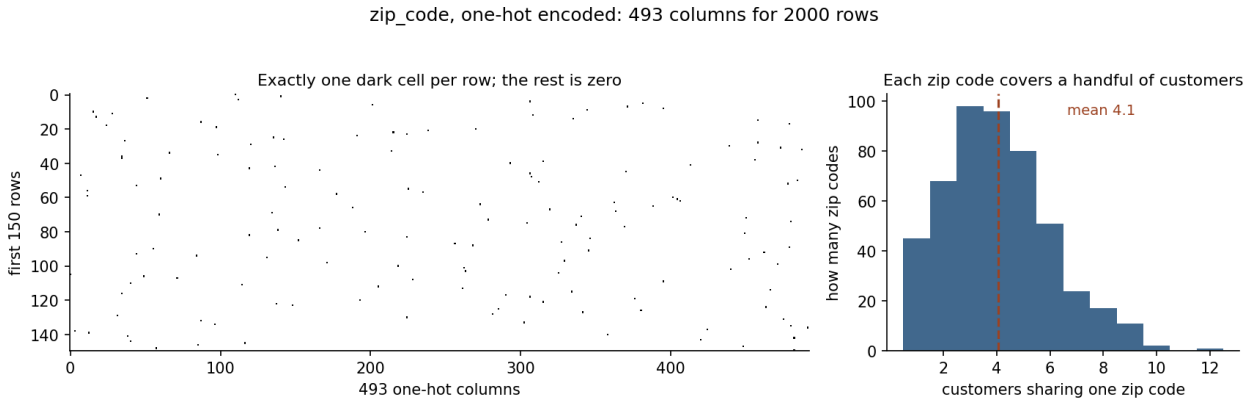
customer	intercept	d_1	d_2	d_3
month-to-month	1	1	0	0
one-year	1	0	1	0
two-year	1	0	0	1
one-year	1	0	1	0

In every row $d_1 + d_2 + d_3 = 1 =$ the intercept, so the four columns are not independent: rank **3**, not 4. Concretely, adding 5 to the intercept and subtracting 5 from all three dummy coefficients changes no prediction at all — infinitely many coefficient vectors fit equally well. Drop d_1 and the three remaining columns have rank 3: unique again, with month-to-month as the reference the other coefficients are measured from.

Slide 34 — The curse of dimensionality, briefly

Deliberately introductory. One-hot encoding `zip_code` adds 493 dimensions to 2,000 examples, with two consequences announced here and developed later. For methods based on distances (lesson 6), in many dimensions all points end up roughly equally far apart, so "nearest" stops meaning much. And more columns mean more coefficients estimated from the same 2,000 rows, so the estimates get noisier and the model can fit noise. Here it mainly explains why someone would be tempted by target encoding.

Slide 35 — zip_code: 493 codes, 2,000 customers



What one-hot encoding `zip_code` *produces. Left: 493 columns, one dark cell per row and nothing else — 99.8% of the matrix is zero. Right: how many zip codes are shared by how many customers; the bar standing at 4 reaches 96, so 96 codes have exactly 4 customers each, and the average is 4.1. A column that carries four rows cannot support a coefficient estimated from those four rows, and that is the curse of dimensionality in its most concrete form.*

Read the right-hand chart carefully. The horizontal axis is **how many customers share a zip code**; the vertical axis is **how many zip codes have that many customers**. Most codes have only 2, 3, 4 or 5 customers; on average $2{,}000/493 \simeq 4.1$. This prepares the target-encoding problem: computing "the churn rate of this zip code" from four customers makes each customer's own label a quarter of the number assigned to them.

Question — about 100 codes with 4 customers?

> *Can one say that about 100 different zip codes have only 4 customers each?*

Yes, phrased this way: "about 100 distinct zip codes appear in 4 rows each, so each is shared by 4 customers". The bar at 4 reaches **96**, so 96 codes have exactly 4 customers, and $96 \times 4 = 384$ of the 2,000 customers belong to one of them. Avoid "100 customers have 4 zip codes", which inverts the meaning of the axes.

Question — "a quarter of the statistic"

> *What does it mean that, if "the churn rate of this zip code" is computed from four customers, each one's label is a quarter of the number assigned to that same customer?*

Take a zip code with four customers whose labels are $[1, 0, 0, 1]$. Its churn rate is $(1+0+0+1)/4 = 0.5$, and naive target encoding writes 0.5 for all four. For the first customer, $y_1 = 1$, that 0.5 is $\frac{y_1+y_2+y_3+y_4}{4}$: their own label enters it with weight exactly $1/4$. Not "about": exactly 25%.

Leave the first customer out and the others give $(0 + 0 + 1)/3 \simeq 0.333$. So their own 1 pushed the encoding up from 0.333 to 0.5. For the second customer, $y_2 = 0$, leaving them out gives $(1 + 0 + 1)/3 \simeq 0.667$: their own 0 pulled it down to 0.5. **Every customer, through their own label, pushes the encoding towards their own class**, and that is the leak. In general the own label weighs $1/n_c$ in the category's average: 25% with 4 customers, 0.1% with 1,000. Small categories are the dangerous ones; with $n_c = 1$ the encoding is the label itself. Slides 49 and 50 make this exact.

Question — the notebook's zip-code summary

> *Notebook 1, section 8 prints the number of distinct codes, the average customers per code and the codes with exactly one customer, using* `zc = df["zip_code"].value_counts()`. *Why is* `zc.mean()` *the average number of customers per zip code? And where does "roughly one in eleven codes seen only once" come from?*

`value_counts()` turns the column into a series with **one entry per distinct zip code, holding how many customers have it**. A column `Z001, Z001, Z002, Z003, Z001, Z002` becomes `Z001: 3, Z002: 2, Z003: 1`. So `zc` no longer holds codes row by row; it holds counts. Then:

- `zc.shape[0]` is the number of entries, so the number of distinct codes: **493**;
- `len(df)` is the number of customers: **2,000**;
- `zc.mean()` is the mean of the counts, total customers divided by distinct codes: $2{,}000/493 \simeq 4.1$. It averages counts, not zip codes;
- `(zc == 1)` is a series of True and False, and `.sum()` counts the True values, since True counts as 1: the number of codes with exactly one customer, **45**.

"One in eleven" refers to the **distinct codes**, not to the customers: 45 of the 493 codes appear once, $45/493 \simeq 0.091$, about 9%, and $493/45 \simeq 11$. The message is that `zip_code` is a high-cardinality, very sparse column: many categories for 2,000 rows, some represented by a single customer.

Part VI — Pipelines and feature engineering

Slide 36 — Preprocessing is part of f

The formula behind the whole lesson is lesson 1's

$$\mathbb{E}_{T\sim\mathcal{D}^m}\big[\hat{R}_T(f)\big] = R(f),$$

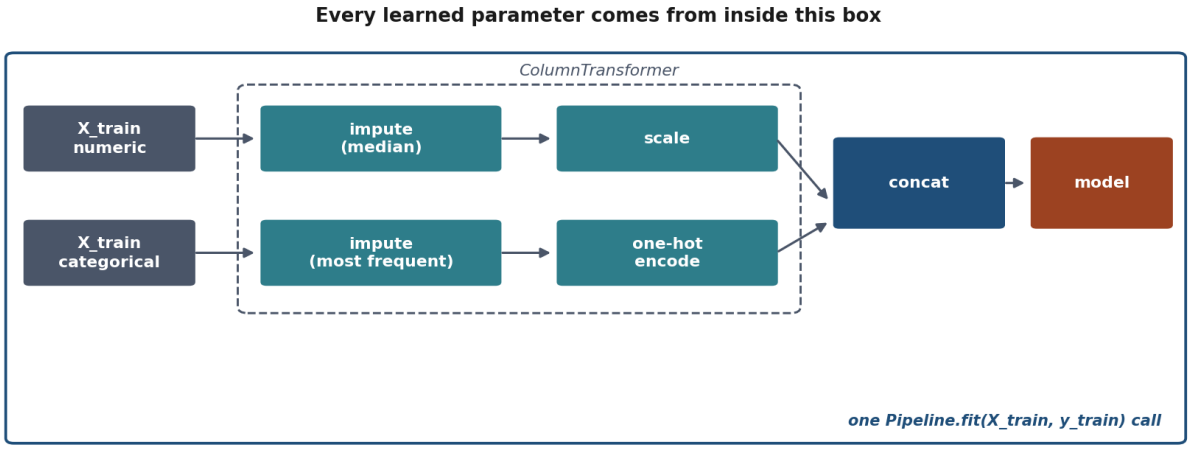
valid as long as f was fixed without looking at the test set T. In words: **the test set measures the system honestly only if the system was not built using it.**

The novelty is what f is. It is not only the classifier; it is the whole chain,

$$f = \text{classifier} \circ \text{encoder} \circ \text{scaler} \circ \text{imputer} \circ \cdots,$$

with every number learned from data, the scaler's means and the imputer's fill values included. If the imputer computed a median using the test set, then f **has seen the test set**, even though the classifier never touched it. That is the theoretical reason for the pipeline.

Slide 37 — ColumnTransformer + Pipeline



The architecture that makes the rule enforceable rather than remembered: numeric and categorical branches, each fitted on training rows only, joined into one object that can be cross-validated whole.

The diagram has two branches. Numeric columns go through imputation and then scaling; categorical columns through imputation and then one-hot encoding. `ColumnTransformer` applies each branch to its own columns and concatenates the results; the outer `Pipeline` passes them to the classifier. One call, `model.fit(X_train, y_train)`, then fits the imputers, the scaler, the encoder and the classifier, in the right order and **on the training data only**. This is not just tidier code: it makes the methodology correct by construction.

Slide 38 — The code

The slide uses short names: `num_pipe`, `cat_pipe`, `num_cols`, `cat_cols`, `clf`. In the notebook:

- `num_pipe` is itself a pipeline of two steps, `SimpleImputer(strategy="median")` and then `StandardScaler()`: raw value → median imputation → standardisation;
- `cat_pipe` is a pipeline of `SimpleImputer(strategy="most_frequent")` and then `OneHotEncoder(drop="first", handle_unknown="ignore")`: raw category → most frequent category → one-hot;
- `num_cols` and `cat_cols` are **lists of column names**, not transformers: `tenure_months`, `monthly_charges`, `age`, `num_support_calls`, and `contract_type`, `region`;
- `clf` is the classifier.

`ColumnTransformer` therefore means: apply `num_pipe` only to `num_cols` and `cat_pipe` only to `cat_cols`, then join the results. Perhaps the most important detail is the last line: **before `fit()`, nothing has been computed from the data.** The lines above only describe the structure.

Question — mean or median, and why the median here

> *What is the difference between the mean and the median, and why is the median chosen here?*

The **mean** uses the value of every observation, $\bar{x} = (x_1 + \cdots + x_n)/n$. The **median** sorts the values and takes the middle one. For $[20, 30, 40, 50, 60]$ both are 40. Add an outlier, $[20, 30, 40, 50, 1000]$:

the mean jumps to 228, the median stays 40. The mean is dragged by an extreme value; the median barely moves.

The choice in the pipeline is deliberate: the outlier section showed billing errors in `monthly_charges`, and the handout says the median is chosen because of what a billing error does to a mean. With charges $[55, 60, 63, 67, 3344]$, a gap filled with the mean gets **717.8**, a value no typical customer pays; the median gives **63**.

That does not make the median always better. On a clean, roughly symmetric column the mean describes the centre well and uses all the quantitative information. The rule is pragmatic: **where a column may hold extreme values, fill gaps with a statistic that resists them.** And the median is learned from the data too: `SimpleImputer` computes it on the training rows, inside the pipeline, never once on the whole dataset.

Slide 39 — The model, on churn

Baseline accuracy 0.806, model accuracy 0.820: a gain of only 0.014. But the area under the receiver operating characteristic curve (AUC) is **0.751**. Read it, for now, as the probability that the model scores a randomly chosen churner above a randomly chosen non-churner: 0.5 is a coin flip, 1 a perfect ranking. Lesson 4 builds it properly.

A precision on the claim that the AUC is "unaffected by the imbalance": more exactly, it does not depend on the share of each class the way accuracy does, because it compares churners with non-churners rather than counting all rows together. It is not immune to every change in the data, such as a shift in who the customers are.

Worked example — the AUC by counting pairs

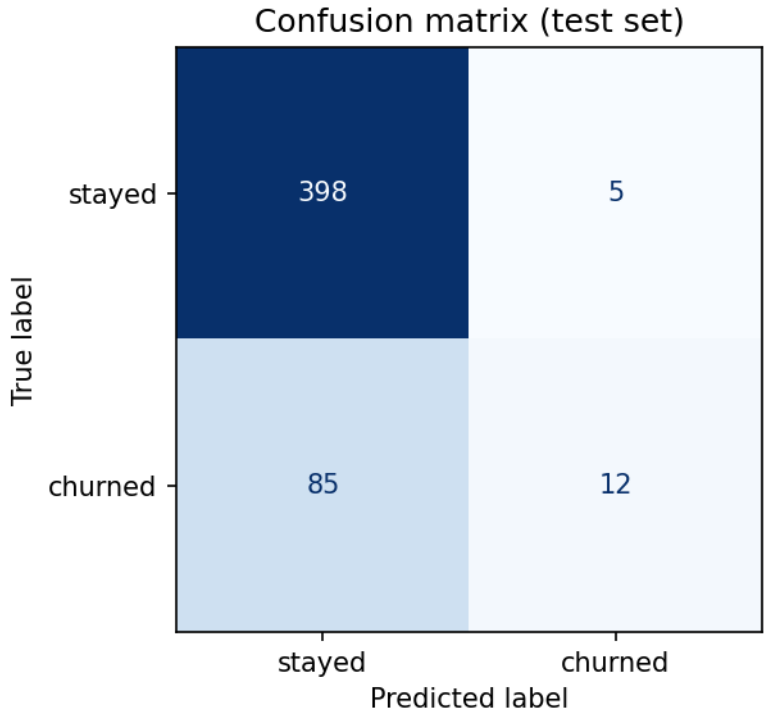
Two churners and three customers who stayed, with the score the model gives each of them. Every pair of one churner and one stayer is a small test: did the model score the churner higher? There are $2 \times 3 = 6$ pairs.

churner's score	above stayer 0.6?	above 0.3?	above 0.2?	pairs right
churner, 0.7	yes	yes	yes	3
churner, 0.4	no	yes	yes	2

5 of 6 pairs are ordered correctly, so the AUC is $5/6 = \mathbf{0.83}$. scikit-learn's `roc_auc_score`, given the five labels and the five scores, returns the same number.

Two readings follow. A model that ordered the customers at random would get about half the pairs right: 0.5. And the majority baseline, which answers "stays" for everyone, gives every customer the same score, so it orders no pair at all: its AUC is 0.5, although its accuracy on the churn data is 0.806. That is why the AUC shows signal that accuracy hides.

Slide 40 — Where the errors fall



The model built on the prepared data. Read the bottom-left cell: the churners it missed. Lesson 4 gives this picture its proper treatment.

A **confusion matrix**: rows are the true class, columns the prediction. Read off the figure, on the 500 test customers:

	predicted stay	predicted churn
actually stayed	398	5
actually churned	85	12

Accuracy is $(398 + 12)/500 = 0.82$, which looks fine. But of the 97 churners the model finds 12: a recall of $12/97 \simeq 0.124$, so about **12% of the customers the retention team wanted to call**. One accuracy figure can hide a model that is close to useless for the actual business goal. (The counts also reproduce the other numbers: 403 customers stayed, so always predicting "stay" scores $403/500 = 0.806$, the baseline.)

Slide 41 — Feature engineering

The new column is total_paid = monthly_charges × tenure_months, on a reasonable hypothesis: what a customer has paid in total may say something that the charge and the tenure do not say separately. The AUC goes from **0.7514** to **0.7548**, a change of +0.0034, and it would be wrong to call it a success: it is tiny.

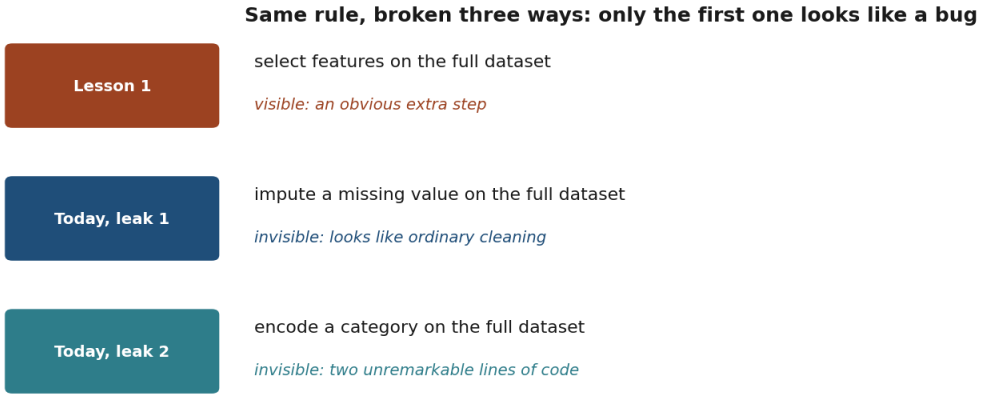
The interesting part comes when `tenure_months` is cleaned first with the domain rule: the AUC becomes **0.7439**, and the "gain" changes sign. A tenure of 999 times an anomalous charge of 3,344.7 gives about 3.34×10^6: the new feature had not discovered anything about customers' spending, it had **amplified the contamination**. Two lessons: feature engineering is a hypothesis, not a guaranteed improvement; and the order **clean, then engineer** can be part of the method.

Slide 42 — Notebook 2, live

The notebook makes the second part of the lesson concrete, and the toolbox table on the slide lists the numbers to come back with: a variance ratio of 110:1 (built as 100:1); a learning rate of 2.0 for scaled features against 0.1 for raw ones; raw features at 2.0 swinging between 0.69 and 8.29; four dummy-plus-intercept columns of rank 3; the full pipeline at an AUC of 0.751; and the engineered feature at $0.7514 \to 0.7548 \to 0.7439$. Spend most time on `ColumnTransformer`: it is the construct you will reuse in every exercise.

Part VII — Leakage in preprocessing

Slide 43 — Same rule, broken three ways



Three ways to break one rule. None of them raises an error, and all three produce a score that is better than the truth.

Three violations of one rule side by side: lesson 1's feature selection before the split, and lesson 2's imputation and target encoding before the split. The last two are more dangerous because they look like plain data cleaning. The common criterion: **no quantity learned from data may use the test set.**

Slide 44 — Leak 1: impute before splitting

`KNNImputer` fills a gap with the average of the five most similar rows, its **donors**. Fit it on the whole dataset before splitting, and the pool of possible donors includes test rows. Test information then flows into the training data:

$$X_{\text{test}} \to X_{\text{train, imputed}} \to \text{model}.$$

Note that y_{test} is never used: using only the **features** of the test rows to build the training set is already enough to break independence. A small case: a training customer has no age, and their two most similar customers are test customers aged 45 and 47; the imputer writes $(45+47)/2 = 46$ into the training row, which now carries information from the test set.

Worked example — the two nearest neighbours are both in the test set

Six customers; C, in training, has no age. `KNNImputer` with $k = 2$ looks for the two customers closest in tenure:

customer	split	tenure (months)	age
A	train	10	34
B	train	30	52
C	train	41	**missing**
D	train	70	61
E	test	42	45
F	test	39	47

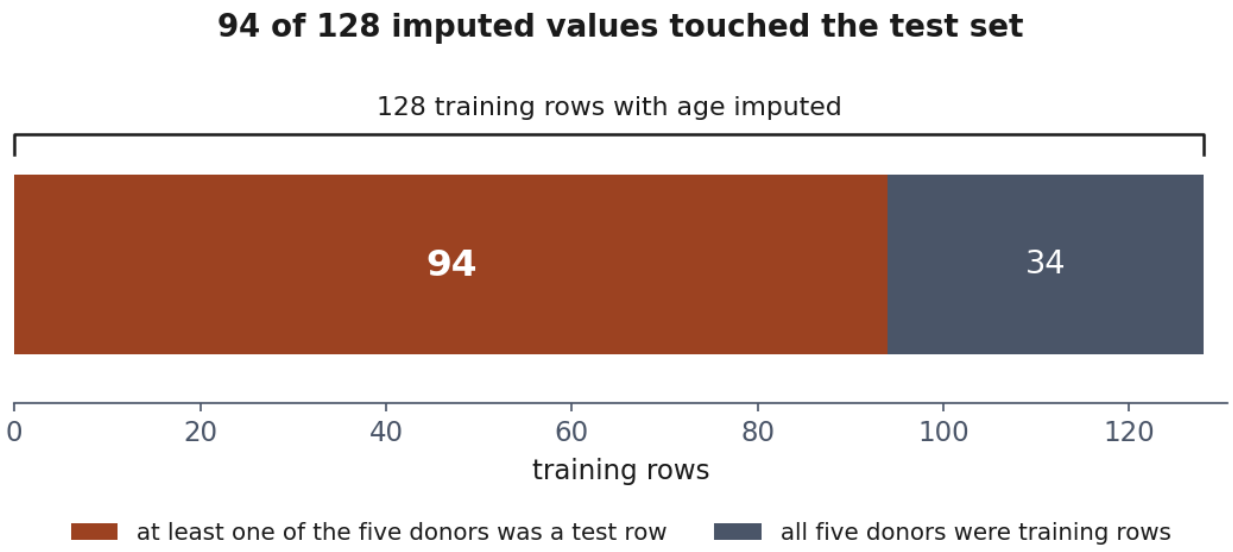
- Fitted on **all six**, C's nearest are E and F (tenure 42 and 39): C gets age **46.0** — built entirely from **test** rows.
- Fitted on the **training rows only**, C's nearest are B and D: age **56.5**.

No test label was touched, and no error was raised. The test customers' features went into a training row, and the model will be graded on those same customers.

Slide 45 — The smoking gun

Notebook 3 rebuilds the donors explicitly, using the same distance as the imputer, `nan_euclidean_distances`. Of the **128** training rows with a missing age, **94** had at least one test row among their five donors: $94/128 \simeq 73\%$. Not a rare or marginal contamination, but almost three quarters of the filled values. And nothing raised an error.

Slide 46 — Leakage, counted



The imputation leak, counted row by row. Of the 128 training rows whose `age` *had to be filled in, 94 borrowed a value from at least one test-set row — the red bar. Not a subtle contamination at the margin: nearly three quarters of every gap this imputer filled was filled with help from data it should never have seen.*

The figure makes the proportion visible: 94 contaminated rows against 34 clean ones. Note that `KNNImputer` did nothing wrong: it searched for neighbours in the set it was given. The bug is in the workflow.

From the notebook, a result worth knowing. The model with the leak scores an AUC of **0.7553**, the honest one **0.7546**: a difference of 0.0007. Repeated over 20 different splits, the difference changes sign, and in **14 of the 20** the leaky model is actually worse. The leak is real, but on this experiment the score cannot reveal it, because `age` carries almost no signal about churn. **"The score did not change" does not mean "there was no leakage."**

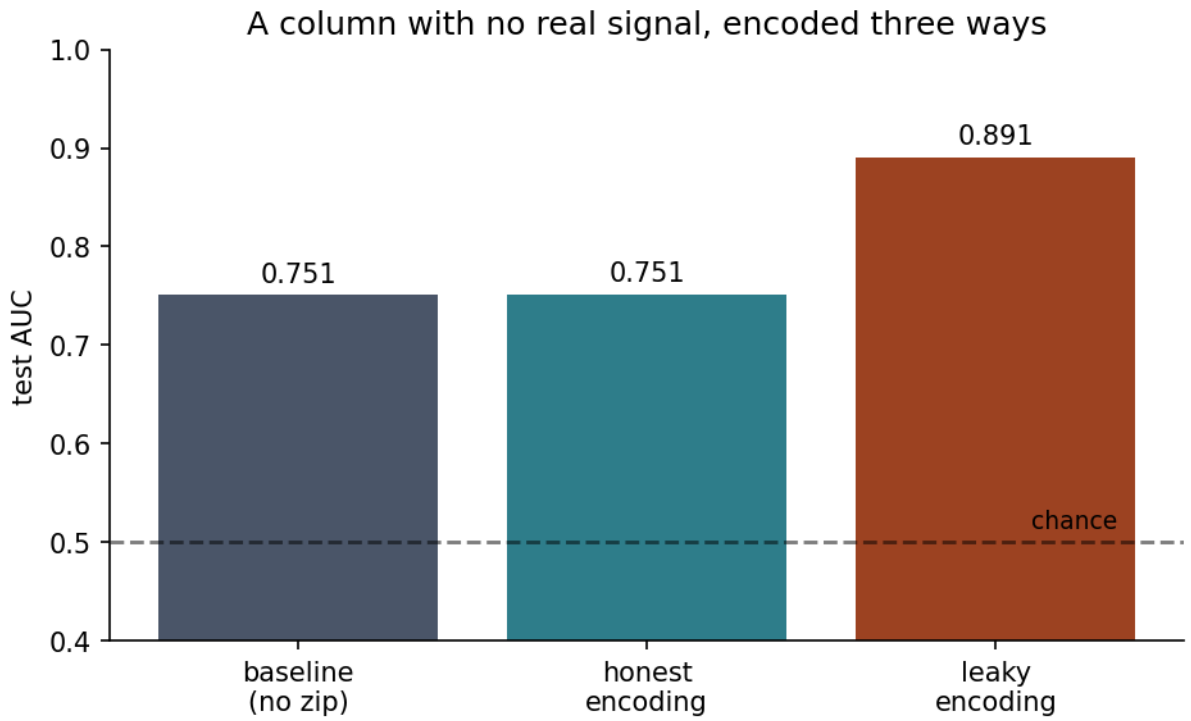
Slide 47 — Leak 2: encode before splitting

Target encoding replaces category c with the average target of its group,

$$\bar{y}_c = \frac{1}{n_c} \sum_{i \in c} y_i.$$

Computed before the split, on all 2,000 customers, the target of a test row takes part in the feature assigned to that same test row: the answer is fed back into the input. The danger is that the result looks completely harmless, a column holding something like `0.25`; nothing in the dataframe says the number was computed from y.

Slide 48 — A column with no real signal, encoded three ways



A column with no real signal, encoded three ways. Target encoding manufactures a predictor out of the labels themselves.

Compare the three numbers directly: without `zip_code`, an AUC of **0.751**; with target encoding done correctly, **0.751**; with target encoding computed before the split, **0.891**. The correct result is the second, because by construction `zip_code` has no relation to churn, so adding it properly should improve nothing. The jump from 0.751 to 0.891 is entirely an artefact of the leak: lesson 1's 77% on coin-flip labels again, produced this time by two unremarkable lines of preprocessing.

Slide 49 — How much of its own label a row sees

The slide worth the most time. Two encodings of row i in category c:

- the **leave-in** encoding $\bar{y}_c$, the category's average with row i included, so y_i contributes to its own feature;
- the **leave-one-out** encoding $\bar{y}_c^{(-i)}$, the average of the other members of the category, with row i left out, so y_i does not contribute.

n_c is the number of rows in category c; it is not n, the number of features.

A small case. A category of four customers with labels $[1, 0, 0, 1]$; take the first, $y_i = 1$. Leave-in: $(1+0+0+1)/4 = 0.5$. Leave-one-out: $(0+0+1)/3 \simeq 0.333$. Letting the customer's own label in raised their feature by $0.5 - 1/3 = 1/6 \simeq 0.167$.

The derivation. Split the leave-in average into row i and the other $n_c - 1$ rows:

$$\bar{y}_c = \frac{y_i + (n_c - 1)\,\bar{y}_c^{(-i)}}{n_c}.$$

Subtract $\bar{y}_c^{(-i)}$, writing it over the same denominator:

$$\bar{y}_c - \bar{y}_c^{(-i)} = \frac{y_i + (n_c - 1)\,\bar{y}_c^{(-i)} - n_c\,\bar{y}_c^{(-i)}}{n_c} = \frac{y_i - \bar{y}_c^{(-i)}}{n_c}.$$

In words: **the artificial contribution of a row's own label is its disagreement with the rest of its group, divided by the size of the group.** In the small case, $(1 - 1/3)/4 = 1/6$, exactly the difference found above. The decisive factor is $1/n_c$: the smaller the group, the more a row sees of its own label.

Worked example — a zip code with no signal, and an AUC of 0.875

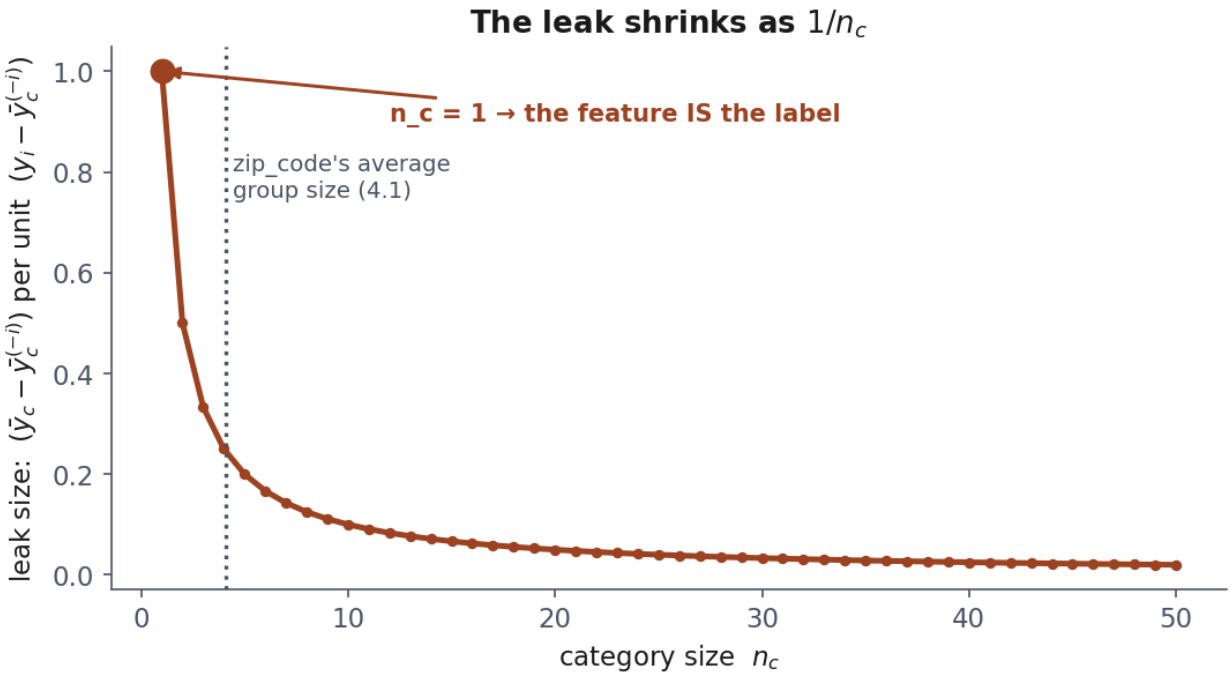
Eight customers in four zip codes, two per code, labels from a coin flip — the code is irrelevant by construction:

zip	churned	target-encoded (all rows)
A	1	1
A	1	1
B	0	0
B	0	0
C	1	1/2
C	0	1/2
D	0	1/2
D	1	1/2

Score each customer by that column and compute the AUC against their own labels: **0.875**. A column that carries nothing ranks churners above stayers, because with two customers per code each customer's own label is half of their own feature.

The slide's identity on one group of four, labels 1, 0, 0, 1, for a churner: with its own label the group mean is 1/2; without it, 1/3; the difference is 1/6 — and indeed $(y_i - \bar{y}^{(-i)})/n_c = (1 - 1/3)/4 = 1/6$.

Slide 50 — The leak shrinks as 1/n_c



The leak is worst where the groups are smallest, falling as $1/n_c$ — which is also where a high-cardinality column keeps most of its categories.

The argument taken to its limit. With $n_c = 1$ the category holds only the customer themself, so $\bar{y}_c = y_i$: **the feature is literally the target**, and the leave-one-out encoding is not even defined, since no one is left. With $n_c = 2$ each label is 50% of the average; with 5, 20%; with 50, about 2%.

This is why high-cardinality columns are the dangerous ones: many categories mean few rows per category. `zip_code` has 493 categories for 2,000 rows, about 4.1 customers each, exactly the danger zone:

high cardinality → small groups → large $1/n_c$ → strong leakage.

Slide 51 — The fix

The fix is not "smooth harder". It is the principle that **a row's label must never contribute to that row's own feature**. `sklearn.preprocessing.TargetEncoder` does this by **cross-fitting**: it splits the training rows into five parts and encodes each row from the statistics of the other four, so y_i is never in the average used for row i. The aim is the same as leave-one-out, done a group of rows at a time instead of recomputing an average for every single row.

Smoothing is a separate idea: for small categories, pull the category's average towards the overall average. It reduces noise and overfitting, but **it cannot repair an encoder fitted on data it should never have seen.** The correct split comes first; smoothing, if wanted, comes after.

Worked example — the same eight customers, cross-fitted

Two folds, one customer of each pair in each. Each customer's code is encoded using **only the other fold** — so only the other member of its pair:

zip	churned	fold	encoded from the other fold
A	1	1	1
A	1	2	1
B	0	1	0
B	0	2	0
C	1	1	0
C	0	2	1
D	0	1	1
D	1	2	0

AUC against the labels: **0.500**. With no shared label, the column is worth nothing — as it should be — where the leaky version scored 0.875. On groups this small the honest score can even fall below 0.5: each customer is described by a neighbour that disagrees with it. That is noise, not signal, and it averages out on real group sizes.

Slide 52 — The rule, restated

The sentence to take away from the lesson: **everything that learns from data is fitted on the training rows only, inside the pipeline.** The slide lists examples that look different: a mean, a median, the nearest neighbours, a churn rate per category, the edges of bins. But the list is not the point. The question to ask of any preprocessing step is: **when it is fitted, does it compute anything from the rows it is given?** If yes, it goes inside the pipeline.

Slide 53 — Notebook 3, live

The notebook closes the circle with two experiments. First, KNN imputation before the split: 94 of 128 training rows used at least one test donor. Second, target encoding of `zip_code`: a column with no signal by construction, which the leaky encoding takes from an AUC of 0.751 to 0.891; `TargetEncoder` inside the `ColumnTransformer` brings it back to 0.751.

The two leaks make an instructive pair. In the imputation leak the methodological violation is large and its effect on the score almost invisible; in the encoding leak the violation produces a large, entirely fictitious improvement. **Both are errors, whatever they do to the score.**

Part VIII — Closing

Slide 54 — What we did today

The pieces put back together: the data was explored before being transformed; for missing values, the percentage turned out to matter less than the mechanism; outliers were separated into unusual and invalid; scaling, encoding and feature engineering were shown to be part of the predictive function; and two leaks were found that look, in the code, like ordinary preprocessing. The thread is lesson 1's rule, generalised: **nothing is learned before the split**, and preprocessing is learning.

Slide 55 — Homework: we discuss it on Friday 16 October

The exercise asks you to build the complete pipeline and to **produce a leak on purpose before fixing it**: that way you see not only how the problem is avoided but which line of code creates it. The data uses a different seed, so the numbers will not match the lesson's exactly. As in Exercise 1, what counts is a methodologically correct workflow and the ability to explain where leakage would occur and why, not the best possible score.

Slide 56 — Before next week

The closing slide sends you to the handout, which carries the derivations behind the slides. Remembering `StandardScaler()`, `OneHotEncoder()` and `Pipeline(...)` is not enough; by the end of the lesson you should be able to explain, without the code:

- why mean imputation weakens a correlation by a factor $\sqrt{1-p}$;
- why Tukey's fence sits at about 2.7σ on a normal column;
- why all k dummies plus an intercept are linearly dependent;
- why, in target encoding, a row sees $1/n_c$ of its own label;
- why a transformation fitted on the test set breaks the independence that makes the test score a valid measurement.

Lesson 3 can then study the **fitting of the model** itself, because the data now reaches the model through a preparation that is consistent and uncontaminated.